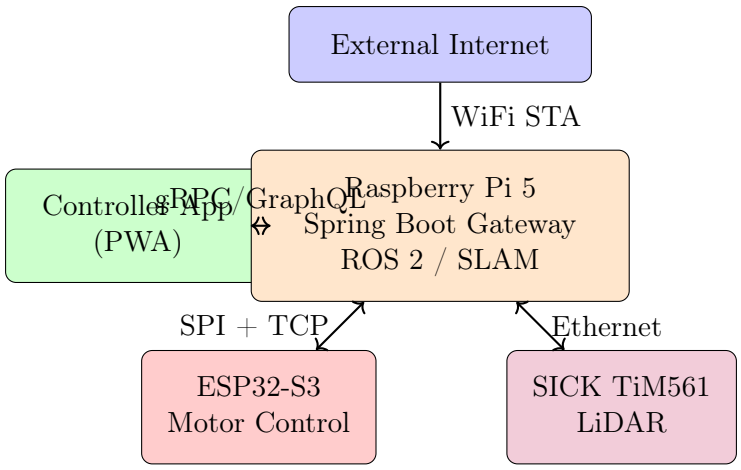
STAR Robot Control System

Software Architecture Primer

Comprehensive Technical Reference for Software Engineers



Version 1.0
December 2025

STAR Development Team
Locked Inc.

Contents

1 Glossary of Key Terms

This section defines key terminology used throughout this document.

1.1 Communication Protocols

gRPC **Google Remote Procedure Call** – A high-performance, open-source RPC framework developed by Google. Uses HTTP/2 for transport and Protocol Buffers (Protobuf) for serialization. Enables efficient binary communication between services with built-in support for streaming.

GraphQL
A query language for APIs developed by Facebook. Unlike REST, clients specify exactly what data they need, reducing over-fetching. Supports queries (read), mutations (write), and subscriptions (real-time updates via WebSocket).

gRPC-Web
A JavaScript implementation of gRPC for browser clients. Since browsers cannot use HTTP/2 trailers directly, gRPC-Web requires a proxy (like Envoy) to translate between gRPC-Web and native gRPC.

Protobuf **Protocol Buffers** – Google's language-neutral, platform-neutral binary serialization format. Smaller and faster than JSON. Messages are defined in `.proto` files and compiled to language-specific code.

WebSocket
A protocol providing full-duplex communication channels over a single TCP connection. Used by GraphQL subscriptions for real-time data streaming from server to client.

REST **Representational State Transfer** – An architectural style for web APIs using HTTP methods (GET, POST, PUT, DELETE). Uses JSON for data exchange. Simpler but less efficient than gRPC for high-frequency communication.

1.2 Middleware and Infrastructure

DDS **Data Distribution Service** – A middleware protocol and API standard for data-centric publish-subscribe communication. Used by ROS 2 for inter-node communication. Provides real-time, scalable, and reliable data exchange with Quality of Service (QoS) policies.

ROS 2 **Robot Operating System 2** – An open-source robotics middleware framework. Provides tools, libraries, and conventions for building robot applications. Uses DDS for communication between nodes.

SLAM **Simultaneous Localization and Mapping** – A computational technique where a robot builds a map of an unknown environment while simultaneously tracking its location within that map.

Nav2 **Navigation 2** – The ROS 2 navigation stack. Provides autonomous navigation capabilities including path planning (A*), obstacle avoidance (DWA), and localization (AMCL).

DWA **Dynamic Window Approach** – A local path planning algorithm used by Nav2. Samples velocities in a "dynamic window" (velocities reachable within one control cycle) and scores trajectories based on proximity to goal, obstacle clearance, and velocity. Runs at 20 Hz in the STAR robot.

AMCL **Adaptive Monte Carlo Localization** – A probabilistic localization algorithm that uses a particle filter to track the robot's pose within a known map. Particles represent hypotheses about the robot's position; sensor observations update particle weights. SLAM Toolbox provides similar functionality during mapping.

mDNS **Multicast DNS** – A protocol that resolves hostnames to IP addresses within small networks without a local DNS server. Enables `star-robot.local` hostname resolution. Implemented by Avahi on Linux.

Avahi A free, open-source implementation of mDNS/DNS-SD (Service Discovery) for Linux. Allows devices to discover each other on a local network without configuration.

Envoy A high-performance, open-source edge and service proxy. Used in this project to translate gRPC-Web requests from browsers to native gRPC for the Spring Boot backend.

1.3 Hardware Interfaces

SPI **Serial Peripheral Interface** – A synchronous serial communication interface for short-distance communication. Uses 4 wires: MOSI (Master Out Slave In), MISO (Master In Slave Out), CLK (Clock), and CS (Chip Select). Used for RPi5-to-ESP32 communication at 10 MHz.

I2C **Inter-Integrated Circuit** – A two-wire serial communication protocol (SDA for data, SCL for clock). Used for connecting sensors like IMUs. Supports multiple devices on the same bus with unique addresses.

UART **Universal Asynchronous Receiver-Transmitter** – A serial communication protocol using TX (transmit) and RX (receive) lines. Commonly used for debugging and some sensor connections.

GPIO **General Purpose Input/Output** – Configurable digital pins on microcontrollers and single-board computers. Can be set as input (read sensors/buttons) or output (control LEDs/relays).

PWM **Pulse Width Modulation** – A technique for controlling analog devices using digital signals. The duty cycle (percentage of time the signal is HIGH) controls the effective power. Used for motor speed control.

Center-Aligned PWM
A PWM mode where the counter counts up then down, centering pulses in the period. Produces cleaner signals with reduced electromagnetic interference (EMI) compared to edge-aligned PWM. Used by the ESP32 MCPWM peripheral for motor control.

PID **Proportional-Integral-Derivative** – A control loop mechanism for maintaining a desired setpoint. The ESP32 runs a 1 kHz PID loop to maintain precise motor velocities based on encoder feedback.

SMBus **System Management Bus** – A two-wire interface based on I2C but with stricter timing requirements and additional features like packet error checking (PEC) and timeouts. Used for battery fuel gauge ICs and power management chips.

1-Wire A serial protocol using a single data wire (plus ground) for communication. Devices are powered parasitically from the data line or via a separate power pin. Used by DS18B20 temperature sensors for thermal monitoring.

1.4 Software Concepts

PWA **Progressive Web App** – A web application that uses modern web technologies to deliver app-like experiences. Can work offline, be installed on devices, and receive push notifications.

Service Worker
A JavaScript worker that runs in the background, separate from the web page. Enables offline functionality by intercepting network requests and serving cached responses.

IndexedDB
A low-level browser API for storing large amounts of structured data. Used for offline telemetry storage in the PWA.

Apollo Client
A comprehensive GraphQL client for JavaScript. Provides caching, state management, and offline support for GraphQL operations.

Workbox A set of JavaScript libraries from Google for adding offline support to web apps. Simplifies service worker creation and caching strategies.

Coroutines
A Kotlin feature for asynchronous programming. Allows writing non-blocking code in a sequential style. Used extensively in the Spring Boot gateway for handling concurrent requests.

Pi4J A Java library for accessing GPIO, I2C, SPI, and other hardware interfaces on Raspberry Pi. Used by the gateway to communicate with the ESP32 via SPI.

FreeRTOS
A real-time operating system kernel for embedded devices. Used by the ESP32 firmware for task scheduling, ensuring the motor control loop runs at exactly 1 kHz.

1.5 Protocols and Formats

COLA-B The communication protocol used by SICK LiDAR sensors (like the TiM561). A binary protocol over TCP/IP for commanding the sensor and receiving scan data.

JSON **JavaScript Object Notation** – A lightweight, human-readable data interchange format. Used by GraphQL and REST APIs. Larger than binary formats like Protobuf.

HTTP/2 The second major version of HTTP. Supports multiplexing (multiple requests over one connection), header compression, and server push. Required by gRPC.

TCP/IP **Transmission Control Protocol / Internet Protocol** – The foundational communication protocols of the internet. Provides reliable, ordered delivery of data packets.

1.6 Deployment and Operations

systemd The init system and service manager for modern Linux distributions. Manages service lifecycle, auto-restart on failure, and dependency ordering.

Buildroot
A tool for building embedded Linux systems. Used to create the minimal custom Linux image for the Raspberry Pi 5.

Prometheus
An open-source monitoring and alerting toolkit. Collects metrics from applications via HTTP endpoints. Used with Spring Boot Actuator for gateway monitoring.

Actuator A Spring Boot module that exposes operational information (health, metrics, info) via HTTP endpoints. Integrates with Prometheus for metrics collection.

JVM **Java Virtual Machine** – The runtime environment for Java and Kotlin applications. The Spring Boot gateway runs on the JVM.

2 Executive Summary

This document provides a comprehensive technical reference for implementing the STAR robot control system software stack. It covers:

- **Protocol Selection**: gRPC + GraphQL hybrid architecture with detailed justification
- **Backend Gateway**: Kotlin/Spring Boot 3 implementation for RPi5
- **Frontend PWA**: Vue 3 + TypeScript Progressive Web App with full offline capability
- **Hardware Integration**: SPI and TCP communication with ESP32
- **Deployment**: Systemd services, mDNS discovery, and metrics

2.1 Architecture Decisions Summary

Decision	Choice	Rationale
Communication Protocol	gRPC + GraphQL Hybrid	Low latency + flexible queries
Backend Framework	Kotlin/Spring Boot 3	JVM performance + coroutines
Frontend Framework	Vue 3 + TypeScript + Vite	Type-safe, reactive UI with Pinia state
Offline Strategy	Workbox + Apollo Cache	Full offline with IndexedDB
ESP32 Communication	SPI (primary) + TCP (fallback)	Redundancy + flexibility
Service Discovery	Avahi/mDNS	Zero-config networking
Metrics	Prometheus + Actuator	Industry standard

Table 1: Key Architecture Decisions

3 Communication Protocol Analysis

3.1 The Problem Statement

The STAR robot requires communication across three distinct paths:

1. **Controller App → RPi5 Gateway**: Over WiFi/Internet (this is the focus)
2. **RPi5 Gateway → ESP32**: Over SPI bus (already defined)
3. **RPi5 Gateway → LiDAR**: Over Ethernet TCP/IP (already defined)

The critical question is: *What protocol should Path #1 use?*

3.2 Protocol Options Evaluated

Four options were evaluated for controller-to-robot communication:

1. gRPC + GraphQL Hybrid
2. ConnectRPC (modern gRPC alternative)
3. WebSocket + GraphQL
4. Pure gRPC with Envoy Proxy

3.3 Detailed Comparison

Aspect	**gRPC + GraphQL**	**ConnectRPC**	**WebSocket + GraphQL**	**Pure gRPC**
Latency	gRPC: 1-5ms, GraphQL: 10-30ms	5-15ms	WS: 5-10ms, GQL: 10-30ms	1-5ms
Packet Size	Binary (gRPC) + JSON (GQL)	Binary OR JSON	JSON only	Binary only
Browser Support	gRPC-Web (limited) + GQL (full)	Full (HTTP/1.1)	Full native	Requires Envoy
Bidirectional	gRPC-Web: NO, GQL Sub: YES	NO	YES (native)	gRPC-Web: NO
Infrastructure	2 servers	1 server	1 server + WS	1 server + Envoy
Learning Curve	High (2 systems)	Medium	Medium	Medium-High
Ecosystem	Very mature (both)	Newer (2022+)	Very mature	Very mature

Table 2: Protocol Comparison Matrix

3.4 Protocol Deep Dive

3.4.1 gRPC + GraphQL Hybrid (Selected)

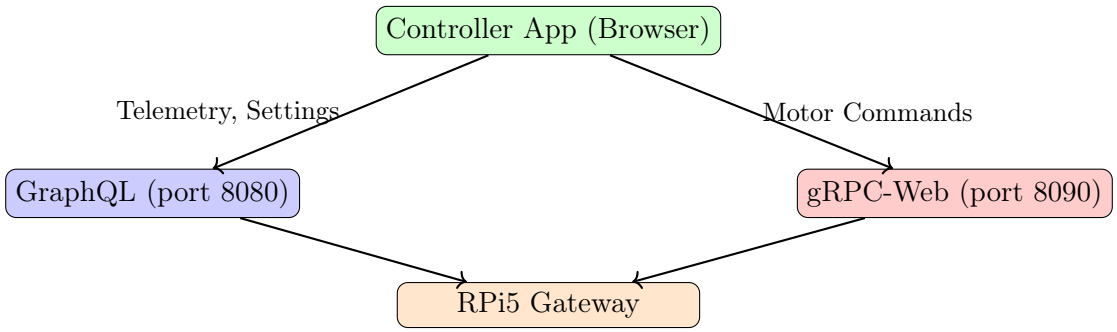


Figure 1: gRPC + GraphQL Hybrid Architecture

Advantages:

- gRPC binary format provides fastest motor command transmission
- GraphQL enables flexible queries for complex UI data needs
- Clear separation of concerns: real-time control vs. data queries
- Both protocols are industry standards with excellent tooling

Disadvantages:

- Two different APIs to maintain
- gRPC-Web requires Envoy proxy as translator
- More complex client code (two client libraries)

3.4.2 Why This Was Selected for STAR

1. **Low-Latency Motor Control**: gRPC's binary Protobuf format minimizes serialization overhead for time-critical motor commands.
2. **Flexible Telemetry Queries**: GraphQL subscriptions provide real-time telemetry streaming with selective field fetching.
3. **Offline Capability**: Apollo Client (GraphQL) has excellent caching and offline support built-in.
4. **Latency Acceptability**: For motor commands, even 10-30ms latency from browser to RPi5 is acceptable because:
   - The ESP32 maintains autonomous 1kHz PID control between commands
   - Human reaction time is approximately 200-300ms
   - The critical real-time path is RPi5 → ESP32 (SPI), not Controller → RPi5

3.5 Packet Format Comparison

3.5.1 Protobuf (gRPC) Binary Encoding

Protobuf uses a compact binary format that excludes field names:

```
message VelocityCommand {
    float left_velocity = 1;    // 4 bytes + tag
    float right_velocity = 2;   // 4 bytes + tag
    uint64 timestamp = 3;       // 1-10 bytes (varint)
}
```

Listing 1: Protobuf Message Definition

Wire format example (VelocityCommand with left=1.0, right=0.5, timestamp=1702500000000):

```
0D 00 00 80 3F  // field 1 (float): 1.0
15 00 00 00 3F  // field 2 (float): 0.5
18 80 A0 CF EE E5 31  // field 3 (varint): timestamp
```

Total size: approximately 17 bytes

3.5.2 JSON (GraphQL) Text Encoding

Equivalent JSON payload:

```
{
  "leftVelocity": 1.0,
  "rightVelocity": 0.5,
  "timestamp": 1702500000000
}
```

Listing 2: JSON Equivalent

Total size: approximately 65 bytes (3.8x larger)

3.5.3 Serialization Performance

Based on industry benchmarks[1]:

Metric	Protobuf	JSON	Difference
Message Size	17 bytes	65 bytes	3.8x smaller
Serialization (JVM)	1.2 μs	4.5 μs	3.75x faster
Deserialization (JVM)	0.8 μs	3.2 μs	4x faster
Network Latency (1Mbps)	0.14 ms	0.52 ms	3.7x faster

Table 3: Protobuf vs JSON Performance Comparison

4 System Architecture

4.1 Network Topology

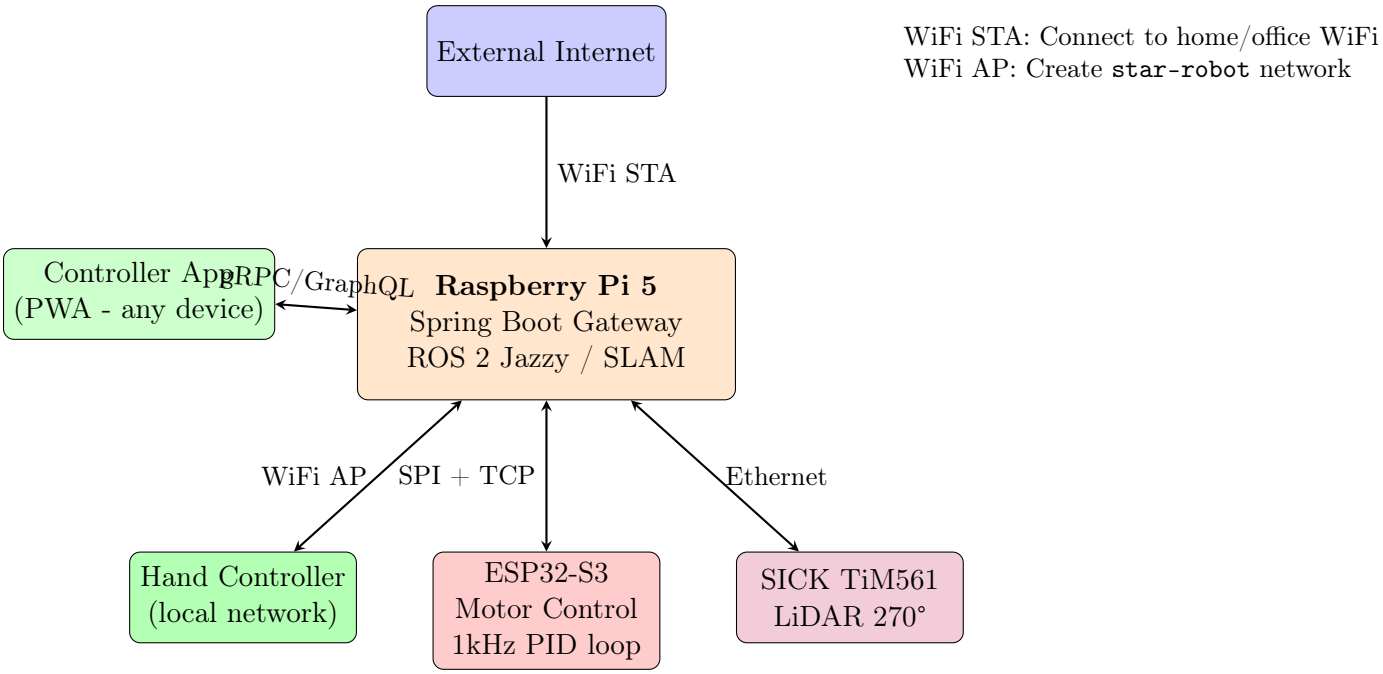


Figure 2: Complete Network Topology

4.2 Dual WiFi Mode Operation

The Raspberry Pi 5 operates in **simultaneous dual WiFi mode**:

1. **Station (STA) Mode**: Connects to external internet for remote access
   - Enables control from anywhere with internet access
   - Allows firmware updates and remote debugging
2. **Access Point (AP) Mode**: Creates local `star-robot` network

[1]Source: Ambassador Labs, Protobuf vs JSON Performance Analysis, 2024

- Provides low-latency local control
- ESP32 connects to this network for TCP fallback
- Hand controller connects for direct operation

4.3 Software Stack Overview

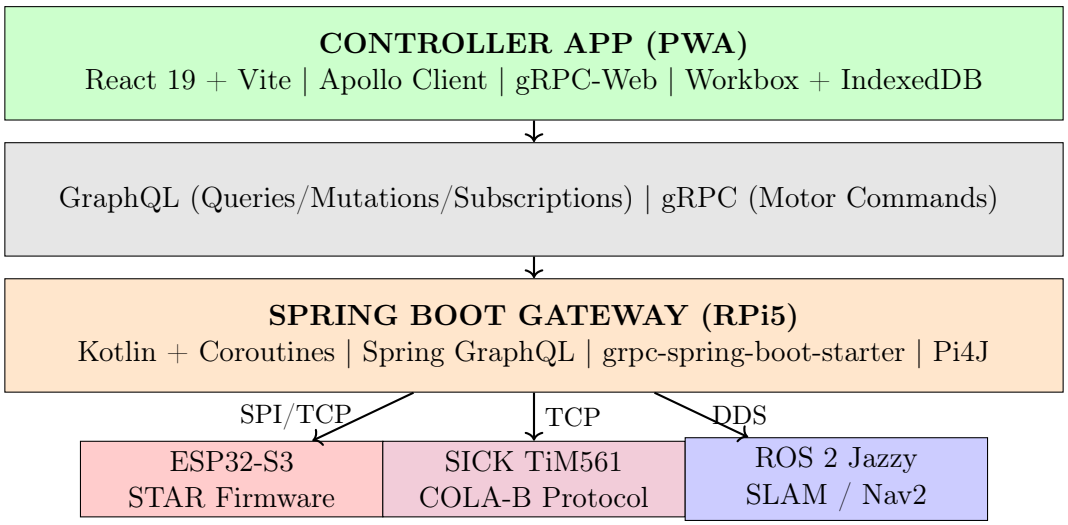


Figure 3: Software Stack Architecture

5 SLAM Algorithm Selection

The STAR robot uses 2D SLAM (Simultaneous Localization and Mapping) for indoor navigation. This section documents the algorithm selection process and rationale.

5.1 Algorithm Comparison

Four SLAM algorithms were evaluated for the STAR platform:

Table 4: SLAM Algorithm Comparison Matrix

Criterion (Weight)	SLAM Toolbox	Cartographer	Hector SLAM	GMapping
Compute Efficiency (0.25)	5	3	4	4
Loop Closure Quality (0.20)	4	5	2	3
Indoor Accuracy (0.25)	5	4	4	4
ROS 2 Support (0.15)	5	4	3	4
Memory Usage (0.15)	4	3	5	4
Weighted Score	**4.55**	**3.75**	**3.55**	**3.80**

Scoring: 1 = Poor, 2 = Below Average, 3 = Average, 4 = Good, 5 = Excellent

5.2 Selection: SLAM Toolbox

SLAM Toolbox was selected as the primary SLAM algorithm based on:

1. **Compute Efficiency** – Optimized for resource-constrained platforms like RPi5. Achieves 10+ Hz map updates with 270-degree LiDAR scans.
2. **Indoor Accuracy** – Sub-centimeter accuracy in structured indoor environments. Excellent performance with the SICK TiM561's 0.33-degree angular resolution.
3. **Native ROS 2 Support** – First-class ROS 2 Humble/Jazzy support. Developed specifically for Nav2 integration.
4. **Flexible Operation Modes**:
   - `mapping` – Create new maps (online SLAM)
   - `localization` – Localize in existing maps (AMCL-like)
   - `lifelong` – Continuous mapping with map updates
5. **Serialization** – Maps can be saved/loaded in multiple formats (PGM, YAML, binary).

5.3 SLAM Toolbox Configuration

```
slam_toolbox:
  ros__parameters:
    # Solver parameters
    solver_plugin: solver_plugins::CeresSolver
    ceres_linear_solver: SPARSE_NORMAL_CHOLESKY
    ceres_preconditioner: SCHUR_JACOBI
    ceres_trust_strategy: LEVENBERG_MARQUARDT
    ceres_dogleg_type: TRADITIONAL_DOGLEG
    ceres_loss_function: None

    # Frame IDs
    odom_frame: odom
    map_frame: map
    base_frame: base_link
    scan_topic: /scan
    use_scan_matching: true
    use_scan_barycenter: true

    # Resolution and range
    resolution: 0.05          # 5cm per cell (matches Nav2 costmap)
    max_laser_range: 10.0     # TiM561 max range
    minimum_travel_distance: 0.3
    minimum_travel_heading: 0.3

    # Loop closure
    do_loop_closing: true
    loop_search_maximum_distance: 3.0
    loop_match_minimum_chain_size: 3
    loop_match_maximum_variance_coarse: 3.0
    loop_match_minimum_response_coarse: 0.35
    loop_match_minimum_response_fine: 0.45

    # Performance tuning for RPi5
    transform_publish_period: 0.05  # 20 Hz TF publish rate
```

```
    map_update_interval: 0.5          # 2 Hz map updates (reduce CPU)
    throttle_scans: 1                 # Process every scan
```

Listing 3: slam_toolbox_params.yaml

5.4 Why Not Other Algorithms?

Cartographer Excellent loop closure but computationally expensive. Requires Lua configuration which adds complexity. Better suited for high-end compute platforms.

Hector SLAM Fast and lightweight but lacks loop closure entirely. Drift accumulates over time in longer sessions. Best for short-duration mapping only.

GMapping Particle filter approach with good accuracy but higher memory usage due to particle storage. ROS 2 port is community-maintained, not official.

5.5 Nav2 DWA Controller Parameters

The Dynamic Window Approach (DWA) local planner requires careful tuning for the STAR robot's physical characteristics:

Table 5: DWA Controller Parameters

Parameter	Value	Description
Velocity Limits		
max_vel_x	0.5 m/s	Maximum forward velocity
min_vel_x	-0.3 m/s	Maximum reverse velocity
max_vel_theta	1.0 rad/s	Maximum angular velocity
min_speed_xy	0.0 m/s	Minimum translational speed
Acceleration Limits		
acc_lim_x	1.0 m/s^2	Linear acceleration limit
acc_lim_theta	2.0 rad/s^2	Angular acceleration limit
decel_lim_x	-1.5 m/s^2	Linear deceleration limit
decel_lim_theta	-2.5 rad/s^2	Angular deceleration limit
Trajectory Scoring		
path_distance_bias	32.0	Weight for staying on global path
goal_distance_bias	24.0	Weight for approaching goal
occdist_scale	0.01	Weight for avoiding obstacles
Costmap		
inflation_radius	0.3 m	Obstacle inflation radius
cost_scaling_factor	3.0	Exponential decay rate

```
controller_server:
  ros__parameters:
    controller_frequency: 20.0  # Hz
    FollowPath:
      plugin: "dwb_core::DWBLocalPlanner"
      # Velocity limits
      max_vel_x: 0.5
```

```
      min_vel_x: -0.3
      max_vel_y: 0.0       # Differential drive - no lateral motion
      min_vel_y: 0.0
      max_vel_theta: 1.0
      min_speed_xy: 0.0
      max_speed_xy: 0.5
      min_speed_theta: 0.0
      # Acceleration limits
      acc_lim_x: 1.0
      acc_lim_y: 0.0
      acc_lim_theta: 2.0
      decel_lim_x: -1.5
      decel_lim_y: 0.0
      decel_lim_theta: -2.5
      # Trajectory generation
      vx_samples: 20
      vy_samples: 1          # Differential drive
      vtheta_samples: 40
      sim_time: 1.5          # Trajectory simulation time
      # Critics (trajectory scoring)
      critics: ["RotateToGoal", "Oscillation", "BaseObstacle",
                "GoalAlign", "PathAlign", "PathDist", "GoalDist"]
      PathAlign.scale: 32.0
      GoalAlign.scale: 24.0
      PathDist.scale: 32.0
      GoalDist.scale: 24.0
      BaseObstacle.scale: 0.01
```

Listing 4: nav2_params.yaml (DWA section)

6 ML Inference Pipeline

The STAR robot uses machine learning for person detection as part of its safety system. This section documents the model selection and inference pipeline.

6.1 Model Selection

Table 6: Person Detection Model Comparison

Model	Size (MB)	mAP@0.5	RPi5 FPS	Quantization
YOLOv8n INT8	3.2	0.37	15-18	INT8
YOLOv5n FP16	7.5	0.35	8-10	FP16
MobileNet-SSD	4.8	0.31	12-15	INT8
EfficientDet-Lite0	4.4	0.33	10-12	INT8

YOLOv8n INT8 was selected for its optimal balance of accuracy and performance on the Raspberry Pi 5.

6.2 Model Architecture

- **Base Model**: YOLOv8n (nano variant)
- **Input Resolution**: 320x320 (reduced from 640x640 for speed)
- **Quantization**: INT8 post-training quantization via TensorFlow Lite
- **Classes**: Single class (person) for focused detection
- **Inference Backend**: TensorFlow Lite with XNNPACK delegate

6.3 Inference Pipeline

```python
import cv2
import numpy as np
from tflite_runtime.interpreter import Interpreter

class PersonDetector:
    def __init__(self, model_path: str = "yolov8n_int8.tflite"):
        # Load TFLite model with XNNPACK delegate for ARM optimization
        self.interpreter = Interpreter(
            model_path=model_path,
            num_threads=4  # Use all 4 Cortex-A76 cores
        )
        self.interpreter.allocate_tensors()

        self.input_details = self.interpreter.get_input_details()
        self.output_details = self.interpreter.get_output_details()
        self.input_shape = self.input_details[0]['shape'][1:3]  # (320, 320)

        # Detection thresholds
        self.confidence_threshold = 0.5
        self.nms_threshold = 0.4

    def preprocess(self, frame: np.ndarray) -> np.ndarray:
        """Resize and normalize input frame."""
        resized = cv2.resize(frame, tuple(self.input_shape))
        # INT8 quantization: scale to 0-255 range
        normalized = resized.astype(np.uint8)
        return np.expand_dims(normalized, axis=0)

    def detect(self, frame: np.ndarray) -> list:
        """Run inference and return person detections."""
        input_data = self.preprocess(frame)
        self.interpreter.set_tensor(
            self.input_details[0]['index'],
            input_data
        )
        self.interpreter.invoke()

        # Parse YOLO output format
        output = self.interpreter.get_tensor(
```

```
            self.output_details[0]['index']
        )
        detections = self._parse_output(output, frame.shape)
        return detections

    def _parse_output(self, output: np.ndarray,
                      original_shape: tuple) -> list:
        """Convert YOLO output to bounding boxes."""
        detections = []
        h, w = original_shape[:2]
        scale_x, scale_y = w / self.input_shape[1], h /
           self.input_shape[0]

        for detection in output[0]:
            confidence = detection[4]
            if confidence < self.confidence_threshold:
                continue

            # Extract bounding box (center format to corner format)
            cx, cy, bw, bh = detection[:4]
            x1 = int((cx - bw / 2) * scale_x)
            y1 = int((cy - bh / 2) * scale_y)
            x2 = int((cx + bw / 2) * scale_x)
            y2 = int((cy + bh / 2) * scale_y)

            detections.append({
                'bbox': (x1, y1, x2, y2),
                'confidence': float(confidence),
                'class': 'person'
            })

        # Apply non-maximum suppression
        return self._nms(detections)

    def _nms(self, detections: list) -> list:
        """Non-maximum suppression to remove overlapping boxes."""
        if not detections:
            return []

        boxes = np.array([d['bbox'] for d in detections])
        scores = np.array([d['confidence'] for d in detections])

        indices = cv2.dnn.NMSBoxes(
            boxes.tolist(),
            scores.tolist(),
            self.confidence_threshold,
            self.nms_threshold
        )

        return [detections[i] for i in indices.flatten()]
```

Listing 5: person_detector.py

6.4 Performance Optimization

1. **XNNPACK Delegate** – ARM-optimized SIMD operations for 2-3x speedup over default TFLite.
2. **Multi-threading** – Utilize all 4 Cortex-A76 cores via `num_threads=4`.
3. **Reduced Resolution** – 320x320 input (vs 640x640) reduces computation by 4x with minimal accuracy loss for person detection.
4. **Frame Skipping** – Process every 2nd frame (15 FPS effective) to maintain system responsiveness.
5. **ROI Processing** – In localization mode, only process regions where persons were previously detected.

6.5 Integration with Safety System

The person detector feeds into the CV Safety System (Section 7):

```
# In main control loop
detector = PersonDetector()
camera = cv2.VideoCapture(0)

while True:
    ret, frame = camera.read()
    if not ret:
        continue

    detections = detector.detect(frame)

    for det in detections:
        # Calculate distance from bounding box size
        # (calibrated for known person height)
        bbox_height = det['bbox'][3] - det['bbox'][1]
        estimated_distance = estimate_distance(bbox_height)

        # Trigger safety response based on distance
        if estimated_distance < EMERGENCY_STOP_DISTANCE:
            robot.emergency_stop()
        elif estimated_distance < SLOW_ZONE_DISTANCE:
            robot.reduce_speed(0.5)  # 50% max speed
```

Listing 6: Integration with safety pipeline

7 CV Safety System

The computer vision safety system provides an additional layer of protection beyond LiDAR-based obstacle detection. It uses the USB camera and YOLOv8n person detector to identify humans and adjust robot behavior accordingly.

7.1 Safety Zones

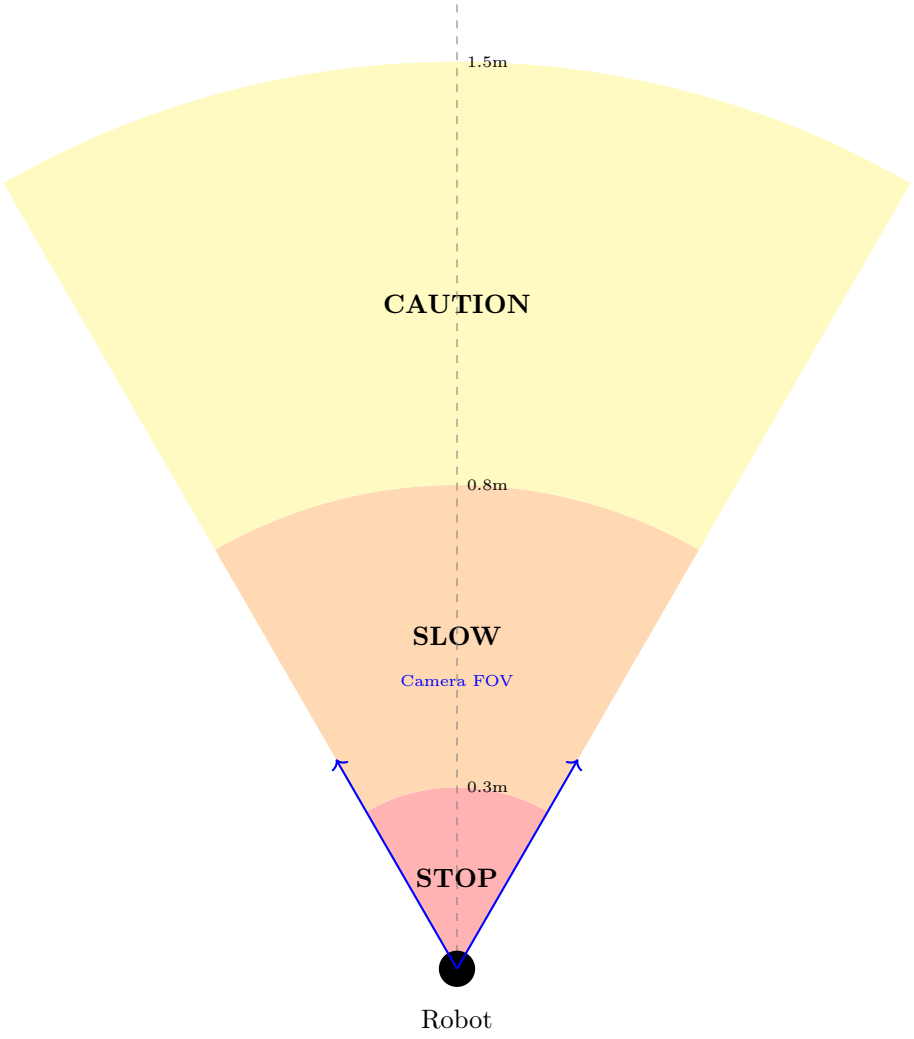


Figure 4: Safety Zone Distances (Top-Down View)

Table 7: Safety Zone Thresholds and Responses

Zone	Distance	Robot Response
Emergency Stop (Red)	< 0.3 m	Immediate stop, hold position
Slow Zone (Orange)	0.3 – 0.8 m	Reduce speed to 50% of max
Caution Zone (Yellow)	0.8 – 1.5 m	Continue at current speed, log warning
Clear (Green)	> 1.5 m	Normal operation

7.2 Distance Estimation

Since the USB camera provides 2D images without depth information, person distance is estimated using the detected bounding box height and a calibrated person height model:

```
# Calibration constants (adjust based on camera focal length and mounting)
```

```
FOCAL_LENGTH_PIXELS = 600  # Camera focal length in pixels
AVERAGE_PERSON_HEIGHT_M = 1.7  # Average adult height in meters
CAMERA_HEIGHT_M = 0.3  # Robot camera mounting height

def estimate_distance(bbox_height_pixels: int,
                      frame_height: int) -> float:
    """
    Estimate distance to person using pinhole camera model.

    distance = (real_height * focal_length) / image_height

    Args:
        bbox_height_pixels: Height of person bounding box in pixels
        frame_height: Total frame height in pixels

    Returns:
        Estimated distance in meters
    """
    if bbox_height_pixels <= 0:
        return float('inf')

    # Pinhole camera model
    distance = (AVERAGE_PERSON_HEIGHT_M * FOCAL_LENGTH_PIXELS) /
       bbox_height_pixels

    # Apply height correction (camera is not at ground level)
    # This is a simplification; more accurate models use full pose
       estimation
    distance = max(0.1, distance)

    return distance

# Calibration procedure:
# 1. Place person at known distance (e.g., 1.0m, 2.0m, 3.0m)
# 2. Record bounding box heights
# 3. Calculate FOCAL_LENGTH_PIXELS = (bbox_height * distance) /
   person_height
# 4. Average across multiple measurements
```

Listing 7: Distance estimation from bounding box

7.3 Safety State Machine

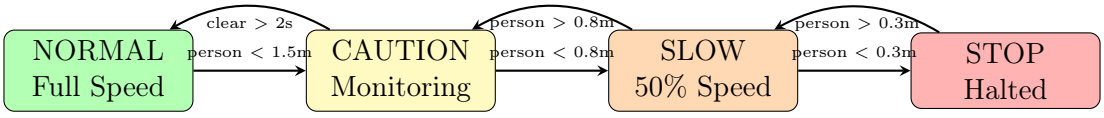


Figure 5: CV Safety State Machine

7.4 Integration with Navigation

The CV safety system overrides navigation commands when necessary:

```python
from enum import Enum
from dataclasses import dataclass
from typing import Optional
import time

class SafetyState(Enum):
    NORMAL = "normal"
    CAUTION = "caution"
    SLOW = "slow"
    STOP = "stop"

@dataclass
class SafetyZones:
    EMERGENCY_STOP: float = 0.3    # meters
    SLOW_ZONE: float = 0.8         # meters
    CAUTION_ZONE: float = 1.5      # meters
    CLEAR_TIMEOUT: float = 2.0     # seconds before returning to normal

class SafetyController:
    def __init__(self):
        self.state = SafetyState.NORMAL
        self.last_detection_time: Optional[float] = None
        self.speed_multiplier = 1.0

    def update(self, person_distance: Optional[float]) -> SafetyState:
        """Update safety state based on nearest person distance."""
        current_time = time.time()

        if person_distance is None:
            # No person detected
            if self.last_detection_time is not None:
                elapsed = current_time - self.last_detection_time
                if elapsed > SafetyZones.CLEAR_TIMEOUT:
                    self._transition_to(SafetyState.NORMAL)
            return self.state

        self.last_detection_time = current_time

        # Determine appropriate state based on distance
        if person_distance < SafetyZones.EMERGENCY_STOP:
            self._transition_to(SafetyState.STOP)
        elif person_distance < SafetyZones.SLOW_ZONE:
            self._transition_to(SafetyState.SLOW)
        elif person_distance < SafetyZones.CAUTION_ZONE:
            self._transition_to(SafetyState.CAUTION)

        return self.state

    def _transition_to(self, new_state: SafetyState):
        if new_state != self.state:
            print(f"Safety state: {self.state.value} ->
                {new_state.value}")
            self.state = new_state
```

```
            self._update_speed_multiplier()

    def _update_speed_multiplier(self):
        multipliers = {
            SafetyState.NORMAL: 1.0,
            SafetyState.CAUTION: 1.0,
            SafetyState.SLOW: 0.5,
            SafetyState.STOP: 0.0,
        }
        self.speed_multiplier = multipliers[self.state]

    def apply_to_velocity(self, cmd_vel: tuple) -> tuple:
        """Apply safety multiplier to velocity command."""
        linear, angular = cmd_vel
        return (linear * self.speed_multiplier,
                angular * self.speed_multiplier)
```

Listing 8: SafetyController class

8 Battery Management

The STAR robot implements a multi-threshold battery management system to ensure safe operation and prevent damage from deep discharge.

8.1 Battery State Thresholds

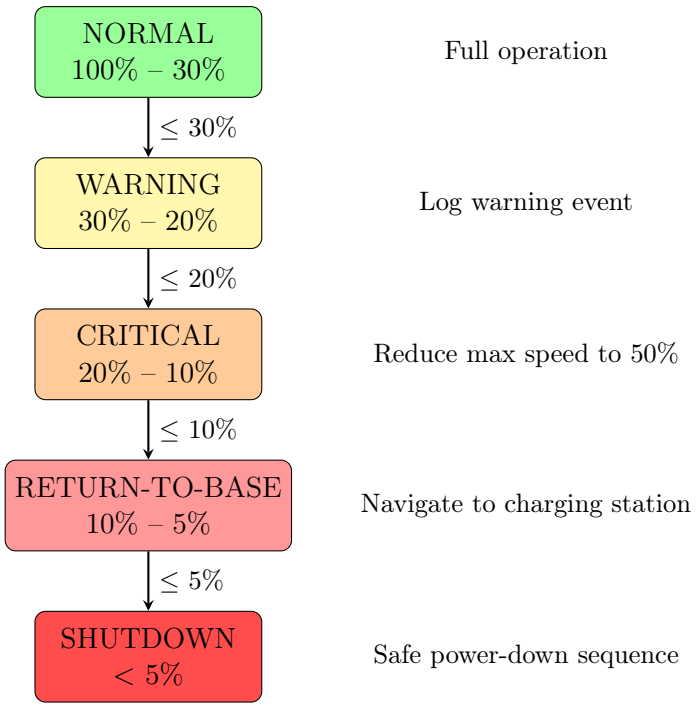


Figure 6: Battery State Machine

Table 8: Battery Management Thresholds

Level	State	Action
100% – 30%	Normal	Full operation, no restrictions
30%	Warning	Log battery warning event to telemetry
20%	Critical	Reduce maximum speed to 50%
10%	Return-to-Base	Abort current task, navigate to charging station
5%	Emergency	Initiate safe shutdown sequence

8.2 Battery Monitoring

Battery voltage and state-of-charge are monitored via the ESP32's ADC with filtering:

```
#include <esp_adc_cal.h>

#define BATTERY_ADC_CHANNEL     ADC1_CHANNEL_6  // GPIO34
#define BATTERY_SAMPLES         16              // Averaging samples
#define VOLTAGE_DIVIDER_RATIO   4.0             // R1=30k, R2=10k
#define BATTERY_MIN_V           10.0            // 3S LiPo empty
#define BATTERY_MAX_V           12.6            // 3S LiPo full

typedef enum {
    BATTERY_NORMAL,
    BATTERY_WARNING,
    BATTERY_CRITICAL,
    BATTERY_RTB,
    BATTERY_SHUTDOWN
} battery_state_t;

static esp_adc_cal_characteristics_t adc_chars;
static battery_state_t current_state = BATTERY_NORMAL;

void battery_init(void) {
    adc1_config_width(ADC_WIDTH_BIT_12);
    adc1_config_channel_atten(BATTERY_ADC_CHANNEL, ADC_ATTEN_DB_11);
    esp_adc_cal_characterize(ADC_UNIT_1, ADC_ATTEN_DB_11,
                             ADC_WIDTH_BIT_12, 1100, &adc_chars);
}

float battery_read_voltage(void) {
    uint32_t adc_sum = 0;

    // Multi-sample averaging for noise reduction
    for (int i = 0; i < BATTERY_SAMPLES; i++) {
        adc_sum += adc1_get_raw(BATTERY_ADC_CHANNEL);
    }
    uint32_t adc_avg = adc_sum / BATTERY_SAMPLES;

    // Convert to voltage
    uint32_t voltage_mv = esp_adc_cal_raw_to_voltage(adc_avg, &adc_chars);

    return (voltage_mv / 1000.0f) * VOLTAGE_DIVIDER_RATIO;
}

```

```
uint8_t battery_get_percentage(float voltage) {
    // Linear approximation (for LiPo, use lookup table for accuracy)
    float pct = (voltage - BATTERY_MIN_V) / (BATTERY_MAX_V -
        BATTERY_MIN_V);
    pct = fmaxf(0.0f, fminf(1.0f, pct));
    return (uint8_t)(pct * 100.0f);
}

battery_state_t battery_update_state(uint8_t percentage) {
    if (percentage <= 5) {
        current_state = BATTERY_SHUTDOWN;
    } else if (percentage <= 10) {
        current_state = BATTERY_RTB;
    } else if (percentage <= 20) {
        current_state = BATTERY_CRITICAL;
    } else if (percentage <= 30) {
        current_state = BATTERY_WARNING;
    } else {
        current_state = BATTERY_NORMAL;
    }
    return current_state;
}
```

Listing 9: Battery monitoring (ESP32)

8.3 Safe Shutdown Sequence

When battery reaches critical level (5%), the system initiates a controlled shutdown:

1. **Stop Motors** – Immediate motor disable, engage brakes if available
2. **Log Position** – Save current map position to persistent storage
3. **Notify User** – Send critical battery notification via GraphQL subscription
4. **Save State** – Persist navigation state, waypoints, and settings to flash
5. **Shutdown Sequence** – Graceful ROS 2 node shutdown, then Linux poweroff

9 Repository Structure

9.1 Directory Layout

```
/STAR/
|-- ESP32-Firmware/           # Existing - keep as-is
|-- star-pi5-os/              # Existing - add gateway to Buildroot
|-- Schematic/                # Existing - hardware designs
|-- Locked_Inc_FTP/           # Existing - documentation
|-- docs/                     # Existing - documentation
|
|-- star-gateway/             # NEW: Spring Boot Gateway
|   |-- build.gradle.kts
|   |-- settings.gradle.kts
```

```
|   |-- src/
|   |   +-- main/
|   |       |-- kotlin/com/star/gateway/
|   |       |   |-- StarGatewayApplication.kt
|   |       |   |-- config/
|   |       |   |   |-- GraphQLConfig.kt
|   |       |   |   |-- GrpcConfig.kt
|   |       |   |   +-- CorsConfig.kt
|   |       |   |-- graphql/
|   |       |   |   |-- TelemetryController.kt
|   |       |   |   |-- SettingsController.kt
|   |       |   |   +-- NavigationController.kt
|   |       |   |-- grpc/
|   |       |   |   +-- MotorControlGrpcService.kt
|   |       |   |-- service/
|   |       |   |   |-- MotorControlService.kt
|   |       |   |   |-- TelemetryService.kt
|   |       |   |   |-- NavigationService.kt
|   |       |   |   +-- SensorFusionService.kt
|   |       |   |-- hardware/
|   |       |   |   |-- Esp32SpiAdapter.kt
|   |       |   |   |-- Esp32TcpAdapter.kt
|   |       |   |   |-- SickLidarAdapter.kt
|   |       |   |   +-- Ros2Bridge.kt
|   |       |   +-- model/
|   |       |       |-- TelemetryData.kt
|   |       |       |-- MotorCommand.kt
|   |       |       +-- NavigationState.kt
|   |       |-- proto/
|   |       |   +-- motor_control.proto
|   |       +-- resources/
|   |           |-- application.yml
|   |           +-- graphql/
|   |               +-- schema.graphqls
|   +-- deployment/
|       |-- star-gateway.service    # Systemd unit
|       +-- avahi/
|           +-- star-robot.service  # mDNS config
|
|-- star-ui/                    # NEW: PWA (moved from UI-Demo)
|   |-- package.json
|   |-- vite.config.js
|   |-- public/
|   |   |-- manifest.json
|   |   +-- sw.js
|   |-- src/
|   |   |-- main.jsx
|   |   |-- App.jsx
|   |   |-- components/           # From UI-Demo
|   |   |-- pages/                # From UI-Demo
|   |   |-- hooks/
|   |   |   |-- useRobotConnection.js
|   |   |   |-- useTelemetry.js
|   |   |   +-- useMotorControl.js
```

```
|   |   |-- graphql/
|   |   |   |-- client.js
|   |   |   |-- queries.js
|   |   |   |-- mutations.js
|   |   |   +-- subscriptions.js
|   |   |-- grpc/
|   |   |   |-- client.js
|   |   |   +-- motor_control_pb.js
|   |   |-- service-worker/
|   |   |   |-- sw.js
|   |   |   +-- offline-cache.js
|   |   +-- storage/
|   |       +-- indexedDB.js
|   +-- workbox-config.js
|
+-- proto/                    # SHARED: Protobuf definitions
    +-- star/
        |-- motor_control.proto
        |-- telemetry.proto
        +-- navigation.proto
```

Listing 10: Complete Repository Structure

10 Spring Boot Gateway Implementation

10.1 Build Configuration

```
plugins {
    kotlin("jvm") version "1.9.22"
    kotlin("plugin.spring") version "1.9.22"
    id("org.springframework.boot") version "3.2.4"
    id("io.spring.dependency-management") version "1.1.4"
    id("com.google.protobuf") version "0.9.4"
}

java {
    sourceCompatibility = JavaVersion.VERSION_17
}

repositories {
    mavenCentral()
}

dependencies {
    // Spring Boot Core
    implementation("org.springframework.boot:spring-boot-starter-webflux")
    implementation("org.springframework.boot:spring-boot-starter-actuator")

    // GraphQL
    implementation("org.springframework.boot:spring-boot-starter-graphql")

    // gRPC
```

```
    implementation("net.devh:grpc-spring-boot-starter:3.1.0.RELEASE")
    implementation("io.grpc:grpc-kotlin-stub:1.4.1")
    implementation("io.grpc:grpc-protobuf:1.60.1")
    implementation("com.google.protobuf:protobuf-kotlin:3.25.2")

    // Kotlin Coroutines
    implementation("org.jetbrains.kotlinx:kotlinx-coroutines-core:1.8.0")
    implementation("org.jetbrains.kotlinx:kotlinx-coroutines-reactor:1.8.0")
    implementation("io.projectreactor.kotlin:reactor-kotlin-extensions")

    // Hardware Access (Pi4J for SPI/GPIO)
    // Pi4J 2.6+ required for Raspberry Pi 5 (RP1 chip support via GpioD)
    implementation("com.pi4j:pi4j-core:2.6.1")
    implementation("com.pi4j:pi4j-plugin-raspberrypi:2.6.1")
    implementation("com.pi4j:pi4j-plugin-gpiod:2.6.1")  // Required for
        RPi5
    implementation("com.pi4j:pi4j-plugin-linuxfs:2.6.1")

    // Metrics
    implementation("io.micrometer:micrometer-registry-prometheus")

    // JSON Processing
    implementation("com.fasterxml.jackson.module:jackson-module-kotlin")

    // Kotlin Reflection
    implementation("org.jetbrains.kotlin:kotlin-reflect")

    // Testing
    testImplementation("org.springframework.boot:spring-boot-starter-test")
    testImplementation("io.projectreactor:reactor-test")
}

protobuf {
    protoc {
        artifact = "com.google.protobuf:protoc:3.25.2"
    }
    plugins {
        id("grpc") {
            artifact = "io.grpc:protoc-gen-grpc-java:1.60.1"
        }
        id("grpckt") {
            artifact = "io.grpc:protoc-gen-grpc-kotlin:1.4.1:jdk8@jar"
        }
    }
    generateProtoTasks {
        all().forEach {
            it.plugins {
                id("grpc")
                id("grpckt")
            }
            it.builtins {
                id("kotlin")
            }
        }
```

```
    }
}

tasks.withType<org.jetbrains.kotlin.gradle.tasks.KotlinCompile> {
    kotlinOptions {
        freeCompilerArgs += "-Xjsr305=strict"
        jvmTarget = "17"
    }
}
```

Listing 11: build.gradle.kts

10.2 Application Configuration

```
spring:
  application:
    name: star-gateway
  graphql:
    graphiql:
      enabled: true
      path: /graphiql
    websocket:
      path: /graphql-ws
    schema:
      locations: classpath:graphql/
  main:
    banner-mode: off

server:
  port: 8080

grpc:
  server:
    port: 9090

# Hardware configuration
star:
  esp32:
    spi:
      enabled: true
      bus: 0
      chip-select: 0
      speed-hz: 10000000  # 10 MHz for motor control
      mode: 0
    tcp:
      enabled: true
      host: 192.168.4.2  # ESP32 IP on AP network
      port: 5000
      timeout-ms: 100
      retry-attempts: 3
  lidar:
    host: 192.168.0.1
    port: 2112
```

```
    protocol: COLA-B
    scan-rate-hz: 15

# Actuator endpoints for health checks
management:
  endpoints:
    web:
      exposure:
        include: health,prometheus,info,metrics
  endpoint:
    health:
      show-details: always
  metrics:
    tags:
      application: star-gateway

logging:
  level:
    com.star.gateway: DEBUG
    org.springframework.graphql: INFO
```

Listing 12: application.yml

10.3 Six-Layer Software Architecture

The STAR system follows a strict six-layer architecture spanning from the remote UI to embedded firmware:

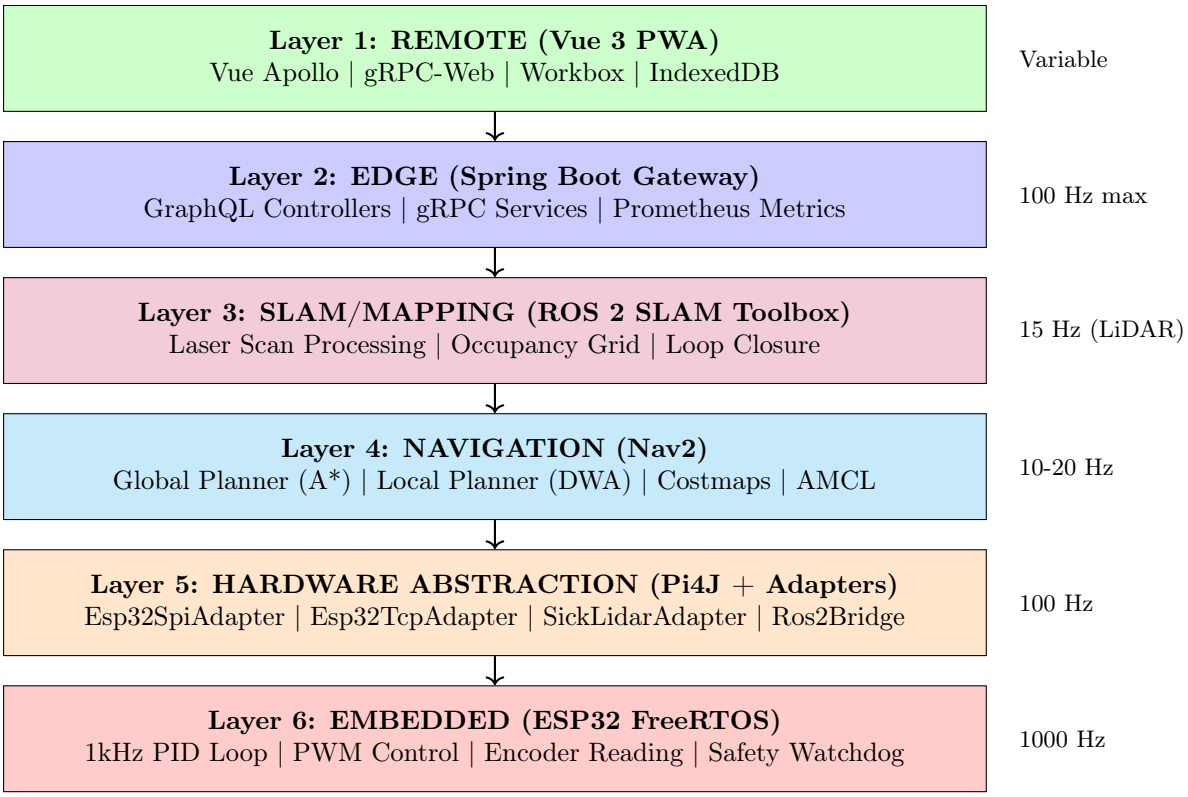


Figure 7: Six-Layer Software Architecture with Update Frequencies

10.3.1 Layer Responsibilities

Layer 1 - Remote (Vue 3 PWA)
User interface running in browser or as installed PWA. Handles user input, displays telemetry, provides offline capability via service workers and IndexedDB.

Layer 2 - Edge (Spring Boot Gateway)
API gateway running on RPi5. Translates between web protocols (GraphQL/gRPC) and robot internals. Manages authentication, rate limiting, and metrics.

Layer 3 - SLAM/Mapping (ROS 2 SLAM Toolbox)
Processes LiDAR scans to build occupancy grid maps. Handles loop closure detection and map optimization. Updates at LiDAR scan rate (15 Hz).

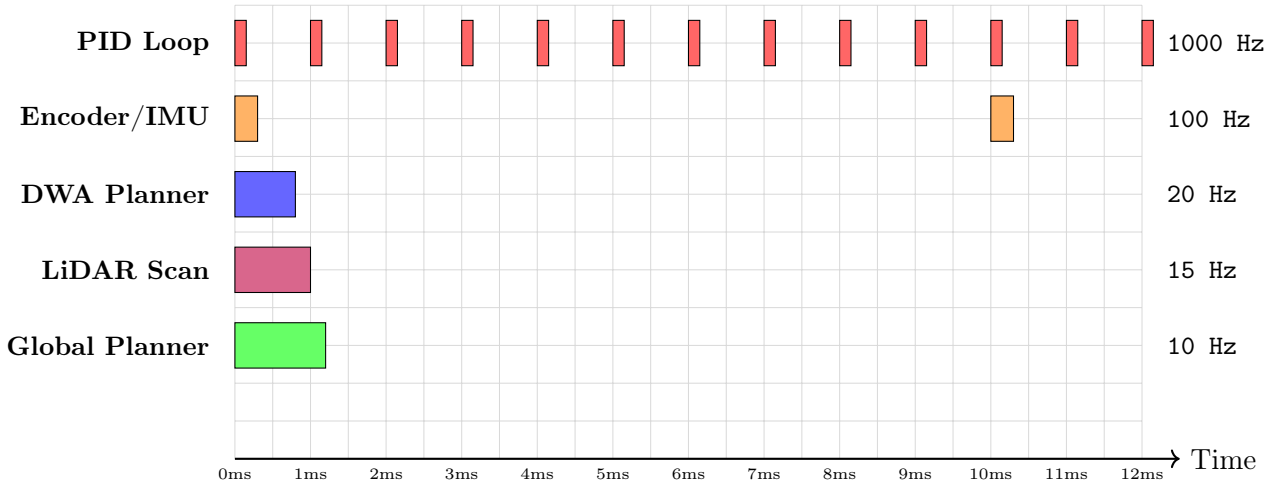
Layer 4 - Navigation (Nav2)
Path planning and obstacle avoidance. Global planner (A*) runs at 10 Hz, local planner (DWA) runs at 20 Hz. Publishes velocity commands to Layer 5.

Layer 5 - Hardware Abstraction (Pi4J + Adapters)
Interfaces between ROS 2 and physical hardware. Communicates with ESP32 via SPI (primary) and TCP (fallback). Reads LiDAR via COLA-B protocol.

Layer 6 - Embedded (ESP32 FreeRTOS)
Real-time motor control at 1 kHz. Runs PID loops, reads encoders, generates PWM signals. Maintains safe operation even if higher layers fail.

10.3.2 Control Timing Hierarchy

The STAR robot operates with a multi-rate control hierarchy, where each layer runs at a frequency appropriate to its function:



Note: Diagram shows 12ms window. Lower frequency tasks extend beyond visible range.

Figure 8: Control Timing Hierarchy (12ms Window)

Table 9: Control Loop Frequencies and Responsibilities

Control Loop	Frequency	Period	Processor
PID Motor Control	1000 Hz	1 ms	ESP32 (FreeRTOS)
Encoder/IMU Sampling	100 Hz	10 ms	ESP32 (FreeRTOS)
DWA Local Planner	20 Hz	50 ms	RPi5 (Nav2)
LiDAR Scan Processing	15 Hz	66.7 ms	RPi5 (ROS 2)
Global Path Planner	10 Hz	100 ms	RPi5 (Nav2)
Telemetry Publishing	10 Hz	100 ms	RPi5 (Spring Boot)
Map Updates	2 Hz	500 ms	RPi5 (SLAM Toolbox)

Timing Design Rationale:

- **1000 Hz PID** – Required for smooth motor control and quick response to disturbances. FreeRTOS ensures deterministic timing.
- **100 Hz Sensor Sampling** – Captures high-frequency vibrations and provides accurate velocity estimates via differentiation.
- **20 Hz DWA** – Fast enough for obstacle avoidance at max velocity (0.5 m/s = 2.5cm per cycle), slow enough to not overload CPU.
- **15 Hz LiDAR** – Fixed by TiM561 hardware. Sufficient for indoor navigation at walking speeds.
- **10 Hz Global Planner** – Path replanning happens infrequently; 10 Hz allows quick rerouting while minimizing CPU load.

10.4 GraphQL Schema

```
# Query operations - read-only data fetching
type Query {
    telemetry: Telemetry!
    systemStatus: SystemStatus!
    settings: Settings!
    navigationState: NavigationState!
    telemetryHistory(limit: Int = 100): [TelemetrySnapshot!]!
}

# Mutation operations - state changes
type Mutation {
    setMode(mode: RobotMode!): Boolean!
    emergencyStop: Boolean!
    setSettings(input: SettingsInput!): Settings!
    addWaypoint(x: Float!, y: Float!): Waypoint!
    clearWaypoints: Boolean!
    startNavigation: Boolean!
    stopNavigation: Boolean!
}

# Subscription operations - real-time streams
type Subscription {
    telemetryStream: Telemetry!
    lidarScan: LidarScan!
    eventLog: Event!
}

# Core telemetry data structure
type Telemetry {
    imu: ImuData!
    gps: GpsData!
    battery: Float!
    wifiSignal: Int!
    cpuUsage: Float!
    temperature: Float!
    motorLoad: Float!
    timestamp: String!
}

type ImuData {
    pitch: Float!
    roll: Float!
    yaw: Float!
    velocity: Float!
    accelerationX: Float!
    accelerationY: Float!
    accelerationZ: Float!
}

type GpsData {
    latitude: Float!
    longitude: Float!
```

```
    altitude: Float!
}

type LidarScan {
    points: [LidarPoint!]!
    timestamp: String!
}

type LidarPoint {
    angle: Float!
    distance: Float!
    intensity: Float!
}

type SystemStatus {
    connectionStatus: ConnectionStatus!
    mode: RobotMode!
    esp32Connected: Boolean!
    lidarConnected: Boolean!
    rosConnected: Boolean!
}

type NavigationState {
    currentPosition: Position!
    path: [Position!]!
    waypoints: [Waypoint!]!
    isNavigating: Boolean!
}

type Position {
    x: Float!
    y: Float!
    heading: Float!
}

type Waypoint {
    id: ID!
    x: Float!
    y: Float!
    label: String
}

type Event {
    timestamp: String!
    message: String!
    type: EventType!
}

type Settings {
    maxSpeed: Float!
    safetyLockEnabled: Boolean!
    autonomousEnabled: Boolean!
    lidarEnabled: Boolean!
}
```

```

input SettingsInput {
    maxSpeed: Float
    safetyLockEnabled: Boolean
    autonomousEnabled: Boolean
    lidarEnabled: Boolean
}

enum RobotMode {
    IDLE
    MANUAL
    AUTONOMOUS
    MAPPING
    EMERGENCY_STOP
}

enum ConnectionStatus {
    CONNECTED
    DISCONNECTED
    CONNECTING
    ERROR
}

enum EventType {
    INFO
    SUCCESS
    WARNING
    ERROR
}
```

Listing 13: schema.graphqls

10.5 gRPC Protobuf Definition

```
syntax = "proto3";

package star.motor;

option java_package = "com.star.gateway.proto";
option java_outer_classname = "MotorControlProto";
option java_multiple_files = true;

// Motor control service for low-latency commands
service MotorControl {
    // Unary RPCs (single request-response)
    rpc SetVelocity(VelocityCommand) returns (CommandResponse);
    rpc EmergencyStop(EmptyRequest) returns (CommandResponse);
    rpc SetMotorPower(MotorPowerCommand) returns (CommandResponse);

    // Server streaming (continuous encoder feedback)
    rpc StreamEncoderData(EmptyRequest) returns (stream EncoderData);
}

```

```
// Velocity command for differential drive
message VelocityCommand {
    float left_velocity = 1;    // m/s, range: -2.0 to 2.0
    float right_velocity = 2;   // m/s, range: -2.0 to 2.0
    uint64 timestamp = 3;       // Unix timestamp in milliseconds
}

// Direct motor power control
message MotorPowerCommand {
    int32 motor_id = 1;         // Motor index: 0-3
    float power = 2;            // Power level: -1.0 to 1.0
}

// Standard command response
message CommandResponse {
    bool success = 1;           // True if command executed
    string message = 2;         // Human-readable status
    uint64 latency_us = 3;      // Round-trip latency in microseconds
}

// Real-time encoder data
message EncoderData {
    int32 motor_id = 1;         // Motor index: 0-3
    int64 ticks = 2;            // Absolute encoder position
    float velocity = 3;         // Current velocity in m/s
    uint64 timestamp = 4;       // Unix timestamp in milliseconds
}

// Empty request placeholder
message EmptyRequest {}
```

Listing 14: motor_control.proto

10.6 Service Implementations

10.6.1 Motor Control Service

```
package com.star.gateway.service

import com.star.gateway.hardware.Esp32SpiAdapter
import com.star.gateway.hardware.Esp32TcpAdapter
import com.star.gateway.model.CommandResult
import io.micrometer.core.instrument.MeterRegistry
import io.micrometer.core.instrument.Timer
import kotlinx.coroutines.async
import kotlinx.coroutines.coroutineScope
import org.slf4j.LoggerFactory
import org.springframework.stereotype.Service
import java.util.concurrent.TimeUnit

@Service
class MotorControlService(
    private val esp32SpiAdapter: Esp32SpiAdapter,
```

```
    private val esp32TcpAdapter: Esp32TcpAdapter,
    private val meterRegistry: MeterRegistry
) {
    private val logger = LoggerFactory.getLogger(javaClass)

    // Metrics
    private val commandCounter =
        meterRegistry.counter("motor.commands.total")
    private val spiErrorCounter =
        meterRegistry.counter("motor.spi.errors")
    private val tcpFallbackCounter =
        meterRegistry.counter("motor.tcp.fallbacks")
    private val latencyTimer: Timer =
        Timer.builder("motor.command.latency")
            .description("Motor command round-trip latency")
            .register(meterRegistry)

    /**
     * Send velocity command to ESP32.
     * Primary: SPI (lowest latency)
     * Fallback: TCP (if SPI fails)
     */
    suspend fun setVelocity(left: Float, right: Float): CommandResult {
        val startTime = System.nanoTime()

        return try {
            val result = esp32SpiAdapter.sendVelocityCommand(left, right)
            commandCounter.increment()
            latencyTimer.record(System.nanoTime() - startTime,
                TimeUnit.NANOSECONDS)
            result
        } catch (e: Exception) {
            logger.warn("SPI failed, falling back to TCP: ${e.message}")
            spiErrorCounter.increment()
            tcpFallbackCounter.increment()

            try {
                esp32TcpAdapter.sendVelocityCommand(left, right)
            } catch (tcpException: Exception) {
                logger.error("TCP fallback also failed:
                    ${tcpException.message}")
                CommandResult.failure("Both SPI and TCP communication
                    failed")
            }
        }
    }

    /**
     * Emergency stop - send to BOTH channels simultaneously for
         redundancy.
     * Returns success if either channel succeeds.
     */
    suspend fun emergencyStop(): CommandResult {
        logger.warn("EMERGENCY STOP triggered")
```

```
63
64          return coroutineScope {
65              val spiDeferred = async {
66                  runCatching { esp32SpiAdapter.emergencyStop() }
67              }
68              val tcpDeferred = async {
69                  runCatching { esp32TcpAdapter.emergencyStop() }
70              }
71
72              val spiResult = spiDeferred.await()
73              val tcpResult = tcpDeferred.await()
74
75              when {
76                  spiResult.isSuccess && tcpResult.isSuccess -> {
77                      logger.info("Emergency stop confirmed on both
                            channels")
78                      CommandResult.success("Emergency stop executed on
                            both channels")
79                  }
80                  spiResult.isSuccess || tcpResult.isSuccess -> {
81                      logger.warn("Emergency stop partially successful")
82                      CommandResult.success("Emergency stop executed
                            (partial)")
83                  }
84                  else -> {
85                      logger.error("Emergency stop FAILED on both
                            channels!")
86                      CommandResult.failure("Emergency stop failed on both
                            channels")
87                  }
88              }
89          }
90      }
91
92      /**
93       * Get current connection status for both channels.
94       */
95      fun getConnectionStatus(): Map<String, Boolean> {
96          return mapOf(
97              "spi" to esp32SpiAdapter.isConnected(),
98              "tcp" to esp32TcpAdapter.isConnected()
99          )
100     }
101 }
```

Listing 15: MotorControlService.kt

10.6.2 ESP32 SPI Adapter

```
1 package com.star.gateway.hardware
2
3 import com.pi4j.Pi4J
4 import com.pi4j.context.Context
```

```
import com.pi4j.io.spi.Spi
import com.pi4j.io.spi.SpiMode
import com.star.gateway.model.CommandResult
import org.slf4j.LoggerFactory
import org.springframework.beans.factory.annotation.Value
import org.springframework.stereotype.Component
import java.nio.ByteBuffer
import java.nio.ByteOrder
import javax.annotation.PostConstruct
import javax.annotation.PreDestroy

@Component
class Esp32SpiAdapter(
    @Value("\${star.esp32.spi.enabled:true}")
    private val enabled: Boolean,

    @Value("\${star.esp32.spi.bus:0}")
    private val spiBus: Int,

    @Value("\${star.esp32.spi.chip-select:0}")
    private val chipSelect: Int,

    @Value("\${star.esp32.spi.speed-hz:10000000}")  // 10 MHz default
    private val speedHz: Int,

    @Value("\${star.esp32.spi.mode:0}")
    private val spiModeValue: Int
) {
    private val logger = LoggerFactory.getLogger(javaClass)

    private var pi4j: Context? = null
    private var spi: Spi? = null
    private var connected = false

    // SPI Command bytes
    companion object {
        const val CMD_SET_VELOCITY: Byte = 0x01
        const val CMD_EMERGENCY_STOP: Byte = 0x02
        const val CMD_GET_ENCODERS: Byte = 0x03
        const val CMD_SET_MOTOR: Byte = 0x04
        const val CMD_PING: Byte = 0x05

        const val RESPONSE_OK: Byte = 0x00
        const val RESPONSE_ERROR: Byte = 0x01
    }

    @PostConstruct
    fun initialize() {
        if (!enabled) {
            logger.info("SPI adapter disabled by configuration")
            return
        }

        try {
```

```
            pi4j = Pi4J.newAutoContext()

            val spiConfig = Spi.newConfigBuilder(pi4j)
                .id("esp32-spi")
                .name("ESP32 Motor Controller")
                .bus(spiBus)
                .chipSelect(chipSelect)
                .baud(speedHz)
                .mode(SpiMode.entries[spiModeValue])
                .build()

            spi = pi4j?.create(spiConfig)
            connected = testConnection()

            logger.info("SPI adapter initialized: bus=$spiBus,
               cs=$chipSelect, speed=$speedHz Hz")
        } catch (e: Exception) {
            logger.error("Failed to initialize SPI adapter: ${e.message}")
            connected = false
        }
    }

    @PreDestroy
    fun shutdown() {
        spi?.close()
        pi4j?.shutdown()
        logger.info("SPI adapter shutdown complete")
    }

    /**
     * Send velocity command.
     * Packet format: [CMD_SET_VELOCITY (1), left_velocity (4),
        right_velocity (4)]
     * Response: [status (1)]
     */
    suspend fun sendVelocityCommand(left: Float, right: Float):
       CommandResult {
        if (!enabled || spi == null) {
            throw IllegalStateException("SPI not initialized")
        }

        val buffer = ByteBuffer.allocate(9)
            .order(ByteOrder.LITTLE_ENDIAN)
            .put(CMD_SET_VELOCITY)
            .putFloat(left)
            .putFloat(right)

        val txData = buffer.array()
        val rxData = ByteArray(2)

        synchronized(this) {
            spi!!.transfer(txData, rxData)
        }

```

```
        return if (rxData[0] == RESPONSE_OK) {
            CommandResult.success("Velocity set: left=$left,
                right=$right")
        } else {
            CommandResult.failure("ESP32 error code: ${rxData[0]}")
        }
    }

    /**
     * Send emergency stop command.
     * Packet format: [CMD_EMERGENCY_STOP (1)]
     * Response: [status (1)]
     */
    suspend fun emergencyStop(): CommandResult {
        if (!enabled || spi == null) {
            throw IllegalStateException("SPI not initialized")
        }

        val txData = byteArrayOf(CMD_EMERGENCY_STOP)
        val rxData = ByteArray(2)

        synchronized(this) {
            spi!!.transfer(txData, rxData)
        }

        return if (rxData[0] == RESPONSE_OK) {
            CommandResult.success("Emergency stop executed")
        } else {
            CommandResult.failure("Emergency stop failed: ${rxData[0]}")
        }
    }

    /**
     * Test SPI connection with ping command.
     */
    private fun testConnection(): Boolean {
        return try {
            val txData = byteArrayOf(CMD_PING)
            val rxData = ByteArray(2)
            spi?.transfer(txData, rxData)
            rxData[0] == RESPONSE_OK
        } catch (e: Exception) {
            logger.warn("SPI connection test failed: ${e.message}")
            false
        }
    }

    fun isConnected(): Boolean = connected && enabled
}
```

Listing 16: Esp32SpiAdapter.kt

10.6.3 ESP32 TCP Adapter

```
package com.star.gateway.hardware

import com.fasterxml.jackson.databind.ObjectMapper
import com.star.gateway.model.CommandResult
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.withContext
import kotlinx.coroutines.withTimeoutOrNull
import org.slf4j.LoggerFactory
import org.springframework.beans.factory.annotation.Value
import org.springframework.stereotype.Component
import java.io.BufferedReader
import java.io.InputStreamReader
import java.io.PrintWriter
import java.net.InetSocketAddress
import java.net.Socket

@Component
class Esp32TcpAdapter(
    @Value("\${star.esp32.tcp.enabled:true}")
    private val enabled: Boolean,

    @Value("\${star.esp32.tcp.host:192.168.4.2}")
    private val host: String,

    @Value("\${star.esp32.tcp.port:5000}")
    private val port: Int,

    @Value("\${star.esp32.tcp.timeout-ms:100}")
    private val timeoutMs: Long,

    @Value("\${star.esp32.tcp.retry-attempts:3}")
    private val retryAttempts: Int,

    private val objectMapper: ObjectMapper
) {
    private val logger = LoggerFactory.getLogger(javaClass)

    data class TcpCommand(
        val command: String,
        val params: Map<String, Any> = emptyMap(),
        val timestamp: Long = System.currentTimeMillis()
    )

    data class TcpResponse(
        val success: Boolean,
        val message: String,
        val data: Map<String, Any>? = null
    )

    /**
     * Send velocity command via TCP.
     */
```

```
    suspend fun sendVelocityCommand(left: Float, right: Float): CommandResult {
        val command = TcpCommand(
            command = "velocity",
            params = mapOf(
                "left" to left,
                "right" to right
            )
        )
        return sendCommand(command)
    }

    /**
     * Send emergency stop via TCP.
     */
    suspend fun emergencyStop(): CommandResult {
        val command = TcpCommand(command = "emergency_stop")
        return sendCommand(command)
    }

    /**
     * Generic command sender with retry logic.
     */
    private suspend fun sendCommand(command: TcpCommand): CommandResult {
        if (!enabled) {
            return CommandResult.failure("TCP adapter disabled")
        }

        var lastException: Exception? = null

        repeat(retryAttempts) { attempt ->
            try {
                return withContext(Dispatchers.IO) {
                    withTimeoutOrNull(timeoutMs) {
                        executeCommand(command)
                    } ?: CommandResult.failure("TCP timeout after ${timeoutMs}ms")
                }
            } catch (e: Exception) {
                lastException = e
                logger.warn("TCP attempt ${attempt + 1}/$retryAttempts failed: ${e.message}")
            }
        }

        return CommandResult.failure(
            "TCP failed after $retryAttempts attempts: ${lastException?.message}"
        )
    }

    private fun executeCommand(command: TcpCommand): CommandResult {
        Socket().use { socket ->
```

```
            socket.connect(InetSocketAddress(host, port),
                timeoutMs.toInt())
            socket.soTimeout = timeoutMs.toInt()

            val writer = PrintWriter(socket.getOutputStream(), true)
            val reader =
                BufferedReader(InputStreamReader(socket.getInputStream()))

            // Send JSON command
            val json = objectMapper.writeValueAsString(command)
            writer.println(json)

            // Read response
            val responseLine = reader.readLine()
            val response = objectMapper.readValue(responseLine,
                TcpResponse::class.java)

            return if (response.success) {
                CommandResult.success(response.message)
            } else {
                CommandResult.failure(response.message)
            }
        }
    }

    /**
     * Check TCP connectivity.
     */
    fun isConnected(): Boolean {
        if (!enabled) return false

        return try {
            Socket().use { socket ->
                socket.connect(InetSocketAddress(host, port),
                    timeoutMs.toInt())
                true
            }
        } catch (e: Exception) {
            false
        }
    }
}
```

Listing 17: Esp32TcpAdapter.kt

10.6.4 GraphQL Controller

```
package com.star.gateway.graphql

import com.star.gateway.model.*
import com.star.gateway.service.TelemetryService
import com.star.gateway.service.MotorControlService
import kotlinx.coroutines.flow.Flow
```

```
import org.springframework.graphql.data.method.annotation.Argument
import org.springframework.graphql.data.method.annotation.MutationMapping
import org.springframework.graphql.data.method.annotation.QueryMapping
import
    org.springframework.graphql.data.method.annotation.SubscriptionMapping
import org.springframework.stereotype.Controller
import reactor.core.publisher.Flux

@Controller
class TelemetryController(
    private val telemetryService: TelemetryService,
    private val motorControlService: MotorControlService
) {
    // ========== QUERIES ==========

    @QueryMapping
    suspend fun telemetry(): Telemetry {
        return telemetryService.getCurrentTelemetry()
    }

    @QueryMapping
    suspend fun systemStatus(): SystemStatus {
        return telemetryService.getSystemStatus()
    }

    @QueryMapping
    suspend fun telemetryHistory(
        @Argument limit: Int?
    ): List<TelemetrySnapshot> {
        return telemetryService.getHistory(limit ?: 100)
    }

    // ========== MUTATIONS ==========

    @MutationMapping
    suspend fun emergencyStop(): Boolean {
        val result = motorControlService.emergencyStop()
        return result.success
    }

    @MutationMapping
    suspend fun setMode(@Argument mode: RobotMode): Boolean {
        return telemetryService.setMode(mode)
    }

    // ========== SUBSCRIPTIONS ==========

    @SubscriptionMapping
    fun telemetryStream(): Flux<Telemetry> {
        return telemetryService.telemetryFlow()
            .asFlux()
    }

    @SubscriptionMapping
```

```
    fun eventLog(): Flux<Event> {
        return telemetryService.eventFlow()
            .asFlux()
    }
}

// Extension function to convert Flow to Flux
private fun <T> Flow<T>.asFlux(): Flux<T> {
    return kotlinx.coroutines.reactor.asFlux(this)
}
```

Listing 18: TelemetryController.kt

10.6.5 gRPC Service

```
package com.star.gateway.grpc

import com.star.gateway.proto.*
import com.star.gateway.service.MotorControlService
import io.grpc.stub.StreamObserver
import kotlinx.coroutines.flow.Flow
import kotlinx.coroutines.flow.flow
import kotlinx.coroutines.runBlocking
import net.devh.boot.grpc.server.service.GrpcService
import org.slf4j.LoggerFactory

@GrpcService
class MotorControlGrpcService(
    private val motorControlService: MotorControlService
) : MotorControlGrpcKt.MotorControlCoroutineImplBase() {

    private val logger = LoggerFactory.getLogger(javaClass)

    /**
     * Handle velocity command via gRPC.
     */
    override suspend fun setVelocity(request: VelocityCommand):
       CommandResponse {
        val startTime = System.nanoTime()

        logger.debug("gRPC SetVelocity: left=${request.leftVelocity},
           right=${request.rightVelocity}")

        val result = motorControlService.setVelocity(
            left = request.leftVelocity,
            right = request.rightVelocity
        )

        val latencyUs = (System.nanoTime() - startTime) / 1000

        return CommandResponse.newBuilder()
            .setSuccess(result.success)
            .setMessage(result.message)
```

```
            .setLatencyUs(latencyUs)
            .build()
    }

    /**
     * Handle emergency stop via gRPC.
     */
    override suspend fun emergencyStop(request: EmptyRequest):
        CommandResponse {
        logger.warn("gRPC EmergencyStop received")

        val result = motorControlService.emergencyStop()

        return CommandResponse.newBuilder()
            .setSuccess(result.success)
            .setMessage(result.message)
            .build()
    }

    /**
     * Stream encoder data to client.
     */
    override fun streamEncoderData(request: EmptyRequest):
        Flow<EncoderData> = flow {
        // Emit encoder data at 100Hz
        while (true) {
            val encoders = motorControlService.getEncoderData()

            encoders.forEach { (motorId, data) ->
                emit(
                    EncoderData.newBuilder()
                        .setMotorId(motorId)
                        .setTicks(data.ticks)
                        .setVelocity(data.velocity)
                        .setTimestamp(System.currentTimeMillis())
                        .build()
                )
            }

            kotlinx.coroutines.delay(10) // 100Hz
        }
    }
}
```

Listing 19: MotorControlGrpcService.kt

11 PWA Frontend Implementation

11.1 Package Configuration

The PWA builds upon the existing UI-Demo codebase with additional dependencies:

```
{
```

```
  "name": "star-ui",
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vue-tsc && vite build",
    "preview": "vite preview",
    "type-check": "vue-tsc --noEmit",
    "generate:proto": "grpc_tools_node_protoc
       --js_out=import_style=commonjs,binary:./src/grpc/generated
       --grpc-web_out=import_style=typescript,mode=grpcwebtext:./src/grpc/generated
       -I ../proto ../proto/star/*.proto"
  },
  "dependencies": {
    // Vue 3 Core
    "vue": "^3.5.13",
    "vue-router": "^4.5.0",
    "pinia": "^2.3.0",

    // UI Libraries
    "chart.js": "^4.4.7",
    "vue-chartjs": "^5.3.2",
    "@vueuse/core": "^12.0.0",
    "lucide-vue-next": "^0.460.0",
    "clsx": "^2.1.1",
    "tailwind-merge": "^3.4.0",

    // GraphQL (Vue Apollo)
    "@vue/apollo-composable": "^4.3.0",
    "@apollo/client": "^3.12.0",
    "graphql": "^16.10.0",
    "graphql-ws": "^5.16.0",
    "apollo3-cache-persist": "^0.15.0",

    // gRPC-Web
    "@improbable-eng/grpc-web": "^0.15.0",
    "google-protobuf": "^3.21.4",

    // Offline storage
    "idb": "^8.0.1",

    // Service Worker
    "workbox-core": "^7.3.0",
    "workbox-precaching": "^7.3.0",
    "workbox-routing": "^7.3.0",
    "workbox-strategies": "^7.3.0"
  },
  "devDependencies": {
    // TypeScript
    "typescript": "~5.7.0",
    "vue-tsc": "^2.2.0",
    "@types/node": "^22.10.0",

    // Vite + Vue
```

```
    "@vitejs/plugin-vue": "^5.2.1",
    "vite": "^6.0.0",

    // Styling
    "tailwindcss": "^4.1.17",
    "autoprefixer": "^10.4.20",

    // PWA plugin
    "vite-plugin-pwa": "^0.21.1",

    // Proto compilation
    "grpc_tools_node_protoc_ts": "^5.3.3",
    "grpc-tools": "^1.12.4"
  }
}
```

Listing 20: package.json (Vue 3 + TypeScript)

11.2 Vite PWA Configuration

```
import { defineConfig } from 'vite'
import vue from '@vitejs/plugin-vue'
import { VitePWA } from 'vite-plugin-pwa'

export default defineConfig({
  plugins: [
    vue(),
    VitePWA({
      registerType: 'autoUpdate',
      includeAssets: [
        'favicon.ico',
        'robots.txt',
        'apple-touch-icon.png'
      ],
      manifest: {
        name: 'STAR Robot Controller',
        short_name: 'STAR',
        description: 'Control interface for STAR LiDAR SLAM robot',
        theme_color: '#500000',  // Texas A&M maroon
        background_color: '#1a1a1a',
        display: 'standalone',
        orientation: 'any',
        start_url: '/',
        scope: '/',
        icons: [
          {
            src: 'pwa-192x192.png',
            sizes: '192x192',
            type: 'image/png'
          },
          {
            src: 'pwa-512x512.png',
            sizes: '512x512',
```

```
            type: 'image/png',
            purpose: 'any maskable'
          }
        ]
      },
      workbox: {
        // Precache static assets
        globPatterns: ['**/*.{js,css,html,ico,png,svg,woff2}'],

        // Runtime caching strategies
        runtimeCaching: [
          {
            // GraphQL API - Network First with fallback
            urlPattern: /^https?:\/\/star-robot\.local(:8080)?\/graphql/,
            handler: 'NetworkFirst',
            options: {
              cacheName: 'graphql-cache',
              expiration: {
                maxEntries: 100,
                maxAgeSeconds: 60 * 60  // 1 hour
              },
              networkTimeoutSeconds: 3,
              cacheableResponse: {
                statuses: [0, 200]
              }
            }
          },
          {
            // Static assets from gateway
            urlPattern: /^https?:\/\/star-robot\.local(:8080)?\/static/,
            handler: 'CacheFirst',
            options: {
              cacheName: 'static-assets',
              expiration: {
                maxEntries: 60,
                maxAgeSeconds: 60 * 60 * 24 * 30  // 30 days
              }
            }
          }
        ]
      },
      devOptions: {
        enabled: true  // Enable PWA in dev mode for testing
      }
    })
  ],
  server: {
    port: 3000,
    proxy: {
      '/graphql': {
        target: 'http://localhost:8080',
        ws: true  // WebSocket proxy for subscriptions
      },
      '/grpc': {
```

```
        target: 'http://localhost:8090'
      }
    }
  }
})
```

Listing 21: vite.config.ts

11.3 Apollo Client with Offline Support (Vue 3)

```
import {
  ApolloClient,
  InMemoryCache,
  HttpLink,
  split,
  ApolloLink,
  type NormalizedCacheObject
} from '@apollo/client/core'
import { GraphQLWsLink } from '@apollo/client/link/subscriptions'
import { getMainDefinition } from '@apollo/client/utilities'
import { createClient } from 'graphql-ws'
import { persistCache, LocalStorageWrapper } from 'apollo3-cache-persist'
import { onError } from '@apollo/client/link/error'
import { provideApolloClient } from '@vue/apollo-composable'

// Gateway URL - defaults to mDNS hostname
const GATEWAY_URL = import.meta.env.VITE_GATEWAY_URL
  || 'http://star-robot.local:8080';

// HTTP link for queries and mutations
const httpLink = new HttpLink({
  uri: `${GATEWAY_URL}/graphql`,
  credentials: 'include'
});

// WebSocket link for subscriptions
const wsLink = new GraphQLWsLink(createClient({
  url: `ws://${GATEWAY_URL.replace(/^https?:\/\//, '')}/graphql-ws`,
  connectionParams: () => ({
    // Add auth tokens here if needed
    timestamp: Date.now()
  }),
  retryAttempts: Infinity,
  shouldRetry: () => true,
  on: {
    connected: () => console.log('WebSocket connected'),
    closed: () => console.log('WebSocket closed'),
    error: (err) => console.error('WebSocket error:', err)
  }
}));

// Error handling link
```

```
const errorLink = onError(({ graphQLErrors, networkError, operation }) =>
   {
  if (graphQLErrors) {
    graphQLErrors.forEach(({ message, locations, path }) => {
      console.error(
        `[GraphQL error]: Message: ${message}, ` +
        `Location: ${locations}, Path: ${path}`
      );
    });
  }

  if (networkError) {
    console.error(`[Network error]: ${networkError}`);
    // Could dispatch to offline queue here
  }
});

// Split link based on operation type
const splitLink = split(
  ({ query }) => {
    const definition = getMainDefinition(query);
    return (
      definition.kind === 'OperationDefinition' &&
      definition.operation === 'subscription'
    );
  },
  wsLink,
  httpLink
);

// Create cache with custom type policies
const cache = new InMemoryCache({
  typePolicies: {
    Query: {
      fields: {
        telemetryHistory: {
          // Merge incoming with existing, keep last 1000
          merge(existing = [], incoming) {
            return [...incoming].slice(-1000);
          }
        }
      }
    },
    Telemetry: {
      // Use timestamp as unique identifier
      keyFields: ['timestamp']
    },
    Waypoint: {
      keyFields: ['id']
    }
  }
});

// Initialize cache persistence
```

```
let apolloClient;

export async function initializeApollo() {
  // Persist cache to localStorage
  await persistCache({
    cache,
    storage: new LocalStorageWrapper(window.localStorage),
    maxSize: 1048576 * 10, // 10MB
    debug: import.meta.env.DEV
  });

  apolloClient = new ApolloClient({
    link: ApolloLink.from([errorLink, splitLink]),
    cache,
    defaultOptions: {
      watchQuery: {
        fetchPolicy: 'cache-and-network',
        errorPolicy: 'all'
      },
      query: {
        fetchPolicy: 'cache-first',
        errorPolicy: 'all'
      },
      mutate: {
        errorPolicy: 'all'
      }
    }
  });

  return apolloClient;
}

export function getApolloClient() {
  if (!apolloClient) {
    throw new Error('Apollo Client not initialized. Call
       initializeApollo() first.');
  }
  return apolloClient;
}
```

Listing 22: src/graphql/client.ts

11.4 GraphQL Operations

```
import { gql } from '@apollo/client';

// ========== QUERIES ==========

export const TELEMETRY_QUERY = gql`
  query GetTelemetry {
    telemetry {
      imu {
        pitch
```

```
        roll
        yaw
        velocity
        accelerationX
        accelerationY
        accelerationZ
      }
      gps {
        latitude
        longitude
        altitude
      }
      battery
      wifiSignal
      cpuUsage
      temperature
      motorLoad
      timestamp
    }
  }
`;

export const SYSTEM_STATUS_QUERY = gql`
  query GetSystemStatus {
    systemStatus {
      connectionStatus
      mode
      esp32Connected
      lidarConnected
      rosConnected
    }
  }
`;

export const NAVIGATION_STATE_QUERY = gql`
  query GetNavigationState {
    navigationState {
      currentPosition {
        x
        y
        heading
      }
      path {
        x
        y
        heading
      }
      waypoints {
        id
        x
        y
        label
      }
      isNavigating
```

```
    }
  }
`;

export const TELEMETRY_HISTORY_QUERY = gql`
  query GetTelemetryHistory($limit: Int) {
    telemetryHistory(limit: $limit) {
      imu {
        pitch
        roll
        yaw
        velocity
      }
      battery
      timestamp
    }
  }
`;

// ========== MUTATIONS ==========

export const EMERGENCY_STOP_MUTATION = gql`
  mutation EmergencyStop {
    emergencyStop
  }
`;

export const SET_MODE_MUTATION = gql`
  mutation SetMode($mode: RobotMode!) {
    setMode(mode: $mode)
  }
`;

export const SET_SETTINGS_MUTATION = gql`
  mutation SetSettings($input: SettingsInput!) {
    setSettings(input: $input) {
      maxSpeed
      safetyLockEnabled
      autonomousEnabled
      lidarEnabled
    }
  }
`;

export const ADD_WAYPOINT_MUTATION = gql`
  mutation AddWaypoint($x: Float!, $y: Float!) {
    addWaypoint(x: $x, y: $y) {
      id
      x
      y
      label
    }
  }
`;
```

```
118
119 // ========== SUBSCRIPTIONS ==========
120
121 export const TELEMETRY_SUBSCRIPTION = gql`
122   subscription TelemetryStream {
123     telemetryStream {
124       imu {
125         pitch
126         roll
127         yaw
128         velocity
129       }
130       battery
131       wifiSignal
132       cpuUsage
133       temperature
134       timestamp
135     }
136   }
137 `;
138
139 export const LIDAR_SCAN_SUBSCRIPTION = gql`
140   subscription LidarScan {
141     lidarScan {
142       points {
143         angle
144         distance
145         intensity
146       }
147       timestamp
148     }
149   }
150 `;
151
152 export const EVENT_LOG_SUBSCRIPTION = gql`
153   subscription EventLog {
154     eventLog {
155       timestamp
156       message
157       type
158     }
159   }
160 `;
```

Listing 23: src/graphql/operations.js

11.5 gRPC-Web Client

```
1 import { grpc } from '@improbable-eng/grpc-web';
2 import {
3   MotorControlClient
4 } from './generated/motor_control_pb_service';
5 import {
```

```
  VelocityCommand,
  EmptyRequest,
  MotorPowerCommand
} from './generated/motor_control_pb';

// gRPC endpoint (through Envoy proxy)
const GRPC_URL = import.meta.env.VITE_GRPC_URL
  || 'http://star-robot.local:8090';

// Create gRPC-Web client
const client = new MotorControlClient(GRPC_URL);

/**
 * Send velocity command via gRPC.
 * @param {number} leftVelocity - Left wheel velocity in m/s
 * @param {number} rightVelocity - Right wheel velocity in m/s
 * @returns {Promise<{success: boolean, message: string, latencyUs:
    number}>}
 */
export function setVelocity(leftVelocity, rightVelocity) {
  return new Promise((resolve, reject) => {
    const request = new VelocityCommand();
    request.setLeftVelocity(leftVelocity);
    request.setRightVelocity(rightVelocity);
    request.setTimestamp(Date.now());

    client.setVelocity(request, {}, (err, response) => {
      if (err) {
        console.error('gRPC setVelocity error:', err);
        reject(err);
      } else {
        resolve({
          success: response.getSuccess(),
          message: response.getMessage(),
          latencyUs: response.getLatencyUs()
        });
      }
    });
  });
}

/**
 * Send emergency stop command via gRPC.
 * @returns {Promise<{success: boolean, message: string}>}
 */
export function emergencyStop() {
  return new Promise((resolve, reject) => {
    const request = new EmptyRequest();

    client.emergencyStop(request, {}, (err, response) => {
      if (err) {
        console.error('gRPC emergencyStop error:', err);
        reject(err);
      } else {
```

```
        resolve({
          success: response.getSuccess(),
          message: response.getMessage()
        });
      }
    });
  });
}

/**
 * Set individual motor power via gRPC.
 * @param {number} motorId - Motor index (0-3)
 * @param {number} power - Power level (-1.0 to 1.0)
 * @returns {Promise<{success: boolean, message: string}>}
 */
export function setMotorPower(motorId, power) {
  return new Promise((resolve, reject) => {
    const request = new MotorPowerCommand();
    request.setMotorId(motorId);
    request.setPower(power);

    client.setMotorPower(request, {}, (err, response) => {
      if (err) {
        reject(err);
      } else {
        resolve({
          success: response.getSuccess(),
          message: response.getMessage()
        });
      }
    });
  });
}

/**
 * Stream encoder data from all motors.
 * @param {Function} onData - Callback for each encoder data packet
 * @param {Function} onError - Callback for errors
 * @param {Function} onEnd - Callback when stream ends
 * @returns {Function} Cancel function to stop the stream
 */
export function streamEncoderData(onData, onError, onEnd) {
  const request = new EmptyRequest();
  const stream = client.streamEncoderData(request);

  stream.on('data', (response) => {
    onData({
      motorId: response.getMotorId(),
      ticks: response.getTicks(),
      velocity: response.getVelocity(),
      timestamp: response.getTimestamp()
    });
  });

```

```
  stream.on('error', (err) => {
    console.error('Encoder stream error:', err);
    onError(err);
  });

  stream.on('end', () => {
    console.log('Encoder stream ended');
    onEnd();
  });

  // Return cancel function
  return () => {
    stream.cancel();
  };
}
```

Listing 24: src/grpc/client.js

11.6 Robot Connection Composable (Vue 3 - Replaces useMockData)

```
import { ref, computed, onMounted, onUnmounted, watch } from 'vue'
import { useQuery, useSubscription, useMutation } from
    '@vue/apollo-composable'
import {
  TELEMETRY_QUERY,
  SYSTEM_STATUS_QUERY,
  NAVIGATION_STATE_QUERY,
  TELEMETRY_SUBSCRIPTION,
  LIDAR_SCAN_SUBSCRIPTION,
  EVENT_LOG_SUBSCRIPTION,
  EMERGENCY_STOP_MUTATION,
  SET_MODE_MUTATION
} from '@/graphql/operations'
import * as grpcClient from '@/grpc/client'
import {
  saveTelemetryToIndexedDB,
  getTelemetryFromIndexedDB,
  saveNavigationState,
  getNavigationState as getStoredNavState
} from '@/storage/indexedDB'
import type { Telemetry, SystemStatus, NavigationState, EncoderData }
    from '@/types'

/**
 * Main composable for robot connection and control.
 * Replaces useMockData with real API integration.
 */
export function useRobotConnection() {
  // Connection state (Vue 3 refs)
  const isOnline = ref(navigator.onLine)
  const lastSyncTime = ref<Date | null>(null)
  const connectionQuality = ref<'good' | 'fair' | 'poor'>('good')

```

```
  // Encoder stream reference
  let encoderStreamCancel: (() => void) | null = null
  const encoderData = ref<Record<number, EncoderData>>({})

  // ========== GraphQL Queries (Vue Apollo) ==========

  const {
    result: telemetryResult,
    loading: telemetryLoading,
    error: telemetryError
  } = useQuery(TELEMETRY_QUERY, () => ({
    pollInterval: isOnline.value ? 0 : 5000,
    fetchPolicy: 'cache-and-network'
  }))

  const { result: statusResult } = useQuery(SYSTEM_STATUS_QUERY, {
    pollInterval: 1000
  })

  const { result: navResult } = useQuery(NAVIGATION_STATE_QUERY, {
    pollInterval: 500
  })

  // ========== GraphQL Subscriptions ==========

  const { result: liveTelemetry } = useSubscription(
    TELEMETRY_SUBSCRIPTION,
    null,
    () => ({ enabled: isOnline.value })
  )

  const { result: lidarResult } = useSubscription(
    LIDAR_SCAN_SUBSCRIPTION,
    null,
    () => ({ enabled: isOnline.value })
  )

  const { result: eventResult } = useSubscription(
    EVENT_LOG_SUBSCRIPTION,
    null,
    () => ({ enabled: isOnline.value })
  )

  // Watch for telemetry updates to persist to IndexedDB
  watch(liveTelemetry, (newData) => {
    if (newData?.telemetryStream) {
      saveTelemetryToIndexedDB(newData.telemetryStream)
      lastSyncTime.value = new Date()
    }
  })

  // ========== GraphQL Mutations ==========

```

```
85  const { mutate: emergencyStopMutation } =
       useMutation(EMERGENCY_STOP_MUTATION)
86  const { mutate: setModeMutation } = useMutation(SET_MODE_MUTATION)
87
88  // ========== Network Status (Vue lifecycle) ==========
89
90  onMounted(() => {
91    const handleOnline = () => {
92      isOnline.value = true
93      console.log('Network: Online')
94    }
95    const handleOffline = () => {
96      isOnline.value = false
97      console.log('Network: Offline')
98    }
99
100   window.addEventListener('online', handleOnline)
101   window.addEventListener('offline', handleOffline)
102 })
103
104 onUnmounted(() => {
105   window.removeEventListener('online', () => {})
106   window.removeEventListener('offline', () => {})
107 })
108
109 // ========== Encoder Stream (gRPC) ==========
110
111 watch(isOnline, (online) => {
112   if (online && !encoderStreamCancel) {
113     encoderStreamCancel = grpcClient.streamEncoderData(
114       (data: EncoderData) => {
115         encoderData.value = {
116           ...encoderData.value,
117           [data.motorId]: data
118         }
119       },
120       (err: Error) => {
121         console.error('Encoder stream error:', err)
122         encoderStreamCancel = null
123       },
124       () => {
125         encoderStreamCancel = null
126       }
127     )
128   }
129 }, { immediate: true })
130
131 onUnmounted(() => {
132   if (encoderStreamCancel) {
133     encoderStreamCancel()
134     encoderStreamCancel = null
135   }
136 })
137
```

```
  // ========== Motor Control (gRPC) ==========

  async function sendMotorCommand(left: number, right: number) {
    if (!isOnline.value) {
      throw new Error('Cannot send motor commands while offline')
    }
    return grpcClient.setVelocity(left, right)
  }

  async function emergencyStop() {
    // Try both gRPC and GraphQL for redundancy
    const results = await Promise.allSettled([
      grpcClient.emergencyStop(),
      emergencyStopMutation()
    ])

    const anySuccess = results.some(
      r => r.status === 'fulfilled' && (r.value as any)?.success !== false
    )

    return { success: anySuccess }
  }

  async function setMode(mode: string) {
    const result = await setModeMutation({ mode })
    return result?.data?.setMode
  }

  // ========== Computed Properties ==========

  const telemetry = computed(() =>
    liveTelemetry.value?.telemetryStream
      || telemetryResult.value?.telemetry
  )

  const lidarScan = computed(() => lidarResult.value?.lidarScan)
  const systemStatus = computed(() => statusResult.value?.systemStatus)
  const navigationState = computed(() => navResult.value?.navigationState)
  const latestEvent = computed(() => eventResult.value?.eventLog)

  // ========== Return Values ==========

  return {
    // Connection state
    isOnline,
    lastSyncTime,
    connectionQuality,

    // Telemetry
    telemetry,
    telemetryLoading,
    telemetryError,

    // LiDAR
```

```
    lidarScan,

    // System status
    systemStatus,

    // Navigation
    navigationState,

    // Events
    latestEvent,

    // Encoder data
    encoderData,

    // Motor control
    sendMotorCommand,
    emergencyStop,
    setMode
  }
}
```

Listing 25: src/composables/useRobotConnection.ts

11.7 IndexedDB Offline Storage

```
import { openDB } from 'idb';

const DB_NAME = 'star-robot-db';
const DB_VERSION = 1;

let dbPromise;

/**
 * Get or create the IndexedDB database.
 */
async function getDB() {
  if (!dbPromise) {
    dbPromise = openDB(DB_NAME, DB_VERSION, {
      upgrade(db, oldVersion, newVersion, transaction) {
        // Telemetry history store
        if (!db.objectStoreNames.contains('telemetry')) {
          const store = db.createObjectStore('telemetry', {
            keyPath: 'id',
            autoIncrement: true
          });
          store.createIndex('timestamp', 'timestamp');
          store.createIndex('savedAt', 'savedAt');
        }

        // Settings store
        if (!db.objectStoreNames.contains('settings')) {
          db.createObjectStore('settings', { keyPath: 'key' });
        }
```

```
29
30          // Navigation state store
31          if (!db.objectStoreNames.contains('navigation')) {
32            db.createObjectStore('navigation', { keyPath: 'key' });
33          }
34
35          // Event log store
36          if (!db.objectStoreNames.contains('events')) {
37            const store = db.createObjectStore('events', {
38              keyPath: 'id',
39              autoIncrement: true
40            });
41            store.createIndex('timestamp', 'timestamp');
42          }
43        }
44      });
45    }
46    return dbPromise;
47  }
48
49  /**
50   * Save telemetry snapshot to IndexedDB.
51   * Automatically prunes to keep only last 10,000 entries.
52   */
53  export async function saveTelemetryToIndexedDB(telemetry) {
54    const db = await getDB();
55
56    await db.add('telemetry', {
57      ...telemetry,
58      savedAt: Date.now()
59    });
60
61    // Prune old entries
62    const count = await db.count('telemetry');
63    if (count > 10000) {
64      const tx = db.transaction('telemetry', 'readwrite');
65      const store = tx.objectStore('telemetry');
66      const index = store.index('savedAt');
67
68      let cursor = await index.openCursor();
69      const toDelete = count - 10000;
70      let deleted = 0;
71
72      while (cursor && deleted < toDelete) {
73        await cursor.delete();
74        cursor = await cursor.continue();
75        deleted++;
76      }
77
78      await tx.done;
79    }
80  }
81
82  /**
```

```
 * Retrieve telemetry history from IndexedDB.
 * @param {number} limit - Maximum entries to return
 * @returns {Promise<Array>} Telemetry entries, newest first
 */
export async function getTelemetryFromIndexedDB(limit = 100) {
  const db = await getDB();
  const tx = db.transaction('telemetry', 'readonly');
  const store = tx.objectStore('telemetry');
  const index = store.index('timestamp');

  const entries = [];
  let cursor = await index.openCursor(null, 'prev');

  while (cursor && entries.length < limit) {
    entries.push(cursor.value);
    cursor = await cursor.continue();
  }

  return entries;
}

/**
 * Save user settings to IndexedDB.
 */
export async function saveSettings(settings) {
  const db = await getDB();
  await db.put('settings', {
    key: 'userSettings',
    ...settings,
    updatedAt: Date.now()
  });
}

/**
 * Retrieve user settings from IndexedDB.
 */
export async function getSettings() {
  const db = await getDB();
  const result = await db.get('settings', 'userSettings');
  return result || null;
}

/**
 * Save navigation state to IndexedDB.
 */
export async function saveNavigationState(state) {
  const db = await getDB();
  await db.put('navigation', {
    key: 'currentState',
    ...state,
    savedAt: Date.now()
  });
}

```

```
/**
 * Retrieve navigation state from IndexedDB.
 */
export async function getNavigationState() {
  const db = await getDB();
  const result = await db.get('navigation', 'currentState');
  return result || null;
}

/**
 * Save event to IndexedDB.
 */
export async function saveEvent(event) {
  const db = await getDB();
  await db.add('events', {
    ...event,
    savedAt: Date.now()
  });

  // Keep only last 500 events
  const count = await db.count('events');
  if (count > 500) {
    const tx = db.transaction('events', 'readwrite');
    const store = tx.objectStore('events');
    const index = store.index('timestamp');

    let cursor = await index.openCursor();
    const toDelete = count - 500;
    let deleted = 0;

    while (cursor && deleted < toDelete) {
      await cursor.delete();
      cursor = await cursor.continue();
      deleted++;
    }
  }
}

/**
 * Get recent events from IndexedDB.
 */
export async function getEvents(limit = 50) {
  const db = await getDB();
  const tx = db.transaction('events', 'readonly');
  const store = tx.objectStore('events');
  const index = store.index('timestamp');

  const entries = [];
  let cursor = await index.openCursor(null, 'prev');

  while (cursor && entries.length < limit) {
    entries.push(cursor.value);
    cursor = await cursor.continue();
  }
```

```
191
192   return entries;
193 }
194
195 /**
196  * Clear all data from IndexedDB.
197  */
198 export async function clearAllData() {
199   const db = await getDB();
200
201   const tx = db.transaction(
202     ['telemetry', 'settings', 'navigation', 'events'],
203     'readwrite'
204   );
205
206   await Promise.all([
207     tx.objectStore('telemetry').clear(),
208     tx.objectStore('settings').clear(),
209     tx.objectStore('navigation').clear(),
210     tx.objectStore('events').clear()
211   ]);
212
213   await tx.done;
214 }
```

Listing 26: src/storage/indexedDB.js

12 ESP32 Firmware Updates

The existing STAR firmware requires additions for TCP communication as a fallback to SPI.

12.1 TCP Server Module

```
1  /**
2   * @file star_tcp_server.h
3   * @brief TCP server for command reception from RPi5
4   *
5   * Provides fallback communication when SPI is unavailable.
6   */
7
8  #ifndef STAR_TCP_SERVER_H
9  #define STAR_TCP_SERVER_H
10
11 #include "esp_err.h"
12
13 #ifdef __cplusplus
14 extern "C" {
15 #endif
16
17 #define TCP_SERVER_PORT 5000
18 #define TCP_MAX_CLIENTS 2
19 #define TCP_RX_BUFFER_SIZE 256
```

```
#define TCP_TX_BUFFER_SIZE 256

/**
 * @brief Motor command callback type
 */
typedef struct {
    float left_velocity;
    float right_velocity;
    bool emergency_stop;
    uint64_t timestamp;
} tcp_motor_command_t;

typedef void (*tcp_command_callback_t)(const tcp_motor_command_t*
   command);

/**
 * @brief TCP server configuration
 */
typedef struct {
    uint16_t port;
    tcp_command_callback_t on_command;
    void* user_context;
} tcp_server_config_t;

/**
 * @brief TCP server handle
 */
typedef struct tcp_server_handle* tcp_server_handle_t;

/**
 * @brief Initialize and start TCP server
 *
 * @param config Server configuration
 * @param out_handle Output handle
 * @return esp_err_t ESP_OK on success
 */
esp_err_t star_tcp_server_init(
    const tcp_server_config_t* config,
    tcp_server_handle_t* out_handle
);

/**
 * @brief Stop and cleanup TCP server
 */
esp_err_t star_tcp_server_deinit(tcp_server_handle_t handle);

/**
 * @brief Get number of connected clients
 */
int star_tcp_server_get_client_count(tcp_server_handle_t handle);

#ifdef __cplusplus
}
#endif
```

```

#endif // STAR_TCP_SERVER_H
```

Listing 27: lib/star_module_tcp/include/star_tcp_server.h

```
/**
 * @file star_tcp_server.c
 * @brief TCP server implementation
 */

#include "star_tcp_server.h"
#include "esp_log.h"
#include "esp_netif.h"
#include "lwip/sockets.h"
#include "cJSON.h"
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"

static const char* TAG = "tcp_server";

struct tcp_server_handle {
    int listen_socket;
    tcp_command_callback_t callback;
    void* user_context;
    TaskHandle_t server_task;
    bool running;
    int client_count;
};

/**
 * @brief Parse JSON command from client
 */
static esp_err_t parse_json_command(
    const char* json_str,
    tcp_motor_command_t* out_cmd
) {
    cJSON* root = cJSON_Parse(json_str);
    if (root == NULL) {
        ESP_LOGE(TAG, "Failed to parse JSON");
        return ESP_ERR_INVALID_ARG;
    }

    memset(out_cmd, 0, sizeof(tcp_motor_command_t));

    cJSON* cmd = cJSON_GetObjectItem(root, "command");
    if (cmd && cJSON_IsString(cmd)) {
        if (strcmp(cmd->valuestring, "velocity") == 0) {
            cJSON* params = cJSON_GetObjectItem(root, "params");
            if (params) {
                cJSON* left = cJSON_GetObjectItem(params, "left");
                cJSON* right = cJSON_GetObjectItem(params, "right");

                if (left && cJSON_IsNumber(left)) {
                    out_cmd->left_velocity = (float)left->valuedouble;
```

```
                }
                if (right && cJSON_IsNumber(right)) {
                    out_cmd->right_velocity = (float)right->valuedouble;
                }
            }
        } else if (strcmp(cmd->valuestring, "emergency_stop") == 0) {
            out_cmd->emergency_stop = true;
        }
    }

    cJSON* ts = cJSON_GetObjectItem(root, "timestamp");
    if (ts && cJSON_IsNumber(ts)) {
        out_cmd->timestamp = (uint64_t)ts->valuedouble;
    }

    cJSON_Delete(root);
    return ESP_OK;
}

/**
 * @brief Build JSON response
 */
static void build_json_response(
    char* buffer,
    size_t size,
    bool success,
    const char* message
) {
    snprintf(buffer, size,
        "{\"success\":%s,\"message\":\"%s\"}",
        success ? "true" : "false",
        message
    );
}

/**
 * @brief Handle single client connection
 */
static void handle_client(
    tcp_server_handle_t handle,
    int client_socket
) {
    char rx_buffer[TCP_RX_BUFFER_SIZE];
    char tx_buffer[TCP_TX_BUFFER_SIZE];

    int len = recv(client_socket, rx_buffer, sizeof(rx_buffer) - 1, 0);

    if (len > 0) {
        rx_buffer[len] = '\0';
        ESP_LOGD(TAG, "Received: %s", rx_buffer);

        tcp_motor_command_t cmd;
        esp_err_t err = parse_json_command(rx_buffer, &cmd);

```

```
        if (err == ESP_OK && handle->callback) {
            handle->callback(&cmd);
            build_json_response(tx_buffer, sizeof(tx_buffer), true, "OK");
        } else {
            build_json_response(tx_buffer, sizeof(tx_buffer), false,
                "Parse error");
        }

        send(client_socket, tx_buffer, strlen(tx_buffer), 0);
    }
}

/**
 * @brief TCP server task
 */
static void tcp_server_task(void* pvParameters) {
    tcp_server_handle_t handle = (tcp_server_handle_t)pvParameters;

    struct sockaddr_in server_addr = {
        .sin_family = AF_INET,
        .sin_port = htons(TCP_SERVER_PORT),
        .sin_addr.s_addr = htonl(INADDR_ANY)
    };

    handle->listen_socket = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);
    if (handle->listen_socket < 0) {
        ESP_LOGE(TAG, "Failed to create socket");
        vTaskDelete(NULL);
        return;
    }

    int opt = 1;
    setsockopt(handle->listen_socket, SOL_SOCKET, SO_REUSEADDR, &opt,
        sizeof(opt));

    if (bind(handle->listen_socket, (struct sockaddr*)&server_addr,
             sizeof(server_addr)) < 0) {
        ESP_LOGE(TAG, "Failed to bind socket");
        close(handle->listen_socket);
        vTaskDelete(NULL);
        return;
    }

    if (listen(handle->listen_socket, TCP_MAX_CLIENTS) < 0) {
        ESP_LOGE(TAG, "Failed to listen");
        close(handle->listen_socket);
        vTaskDelete(NULL);
        return;
    }

    ESP_LOGI(TAG, "TCP server listening on port %d", TCP_SERVER_PORT);

    while (handle->running) {
        struct sockaddr_in client_addr;
```

```
        socklen_t addr_len = sizeof(client_addr);

        int client_socket = accept(
            handle->listen_socket,
            (struct sockaddr*)&client_addr,
            &addr_len
        );

        if (client_socket >= 0) {
            handle->client_count++;
            ESP_LOGI(TAG, "Client connected from %s",
                inet_ntoa(client_addr.sin_addr));

            handle_client(handle, client_socket);

            close(client_socket);
            handle->client_count--;
        }
    }

    close(handle->listen_socket);
    vTaskDelete(NULL);
}

esp_err_t star_tcp_server_init(
    const tcp_server_config_t* config,
    tcp_server_handle_t* out_handle
) {
    if (config == NULL || out_handle == NULL) {
        return ESP_ERR_INVALID_ARG;
    }

    tcp_server_handle_t handle = calloc(1, sizeof(struct
        tcp_server_handle));
    if (handle == NULL) {
        return ESP_ERR_NO_MEM;
    }

    handle->callback = config->on_command;
    handle->user_context = config->user_context;
    handle->running = true;
    handle->client_count = 0;

    BaseType_t ret = xTaskCreate(
        tcp_server_task,
        "tcp_server",
        4096,
        handle,
        5,
        &handle->server_task
    );

    if (ret != pdPASS) {
        free(handle);
```

```
        return ESP_FAIL;
    }

    *out_handle = handle;
    return ESP_OK;
}

esp_err_t star_tcp_server_deinit(tcp_server_handle_t handle) {
    if (handle == NULL) {
        return ESP_ERR_INVALID_ARG;
    }

    handle->running = false;

    // Wake up the accept() call
    if (handle->listen_socket >= 0) {
        shutdown(handle->listen_socket, SHUT_RDWR);
    }

    // Wait for task to finish
    vTaskDelay(pdMS_TO_TICKS(100));

    free(handle);
    return ESP_OK;
}

int star_tcp_server_get_client_count(tcp_server_handle_t handle) {
    return handle ? handle->client_count : 0;
}
```

Listing 28: lib/star_module_tcp/src/star_tcp_server.c

12.2 Main Application Integration

Add TCP server to the main application:

```
// Add to includes
#include "star_tcp_server.h"

// Add global handle
static tcp_server_handle_t g_tcp_server = NULL;

// Add command callback
static void on_tcp_command(const tcp_motor_command_t* cmd) {
    if (cmd->emergency_stop) {
        // Execute emergency stop
        motor_control_emergency_stop();
    } else {
        // Set motor velocities
        motor_control_set_velocity(cmd->left_velocity,
            cmd->right_velocity);
    }
}

```

```
// Add to app_main() after WiFi is connected
void app_main(void) {
    // ... existing initialization ...

    // Initialize TCP server for RPi5 communication
    tcp_server_config_t tcp_config = {
        .port = TCP_SERVER_PORT,
        .on_command = on_tcp_command,
        .user_context = NULL
    };

    esp_err_t err = star_tcp_server_init(&tcp_config, &g_tcp_server);
    if (err == ESP_OK) {
        ESP_LOGI(TAG, "TCP server started on port %d", TCP_SERVER_PORT);
    } else {
        ESP_LOGE(TAG, "Failed to start TCP server");
    }

    // ... rest of initialization ...
}
```

Listing 29: Diff for ESP32-Firmware/src/main.c

13 Process Flow Diagrams

This section illustrates the data flow through the STAR robot system for key operations.

13.1 Motor Command Flow

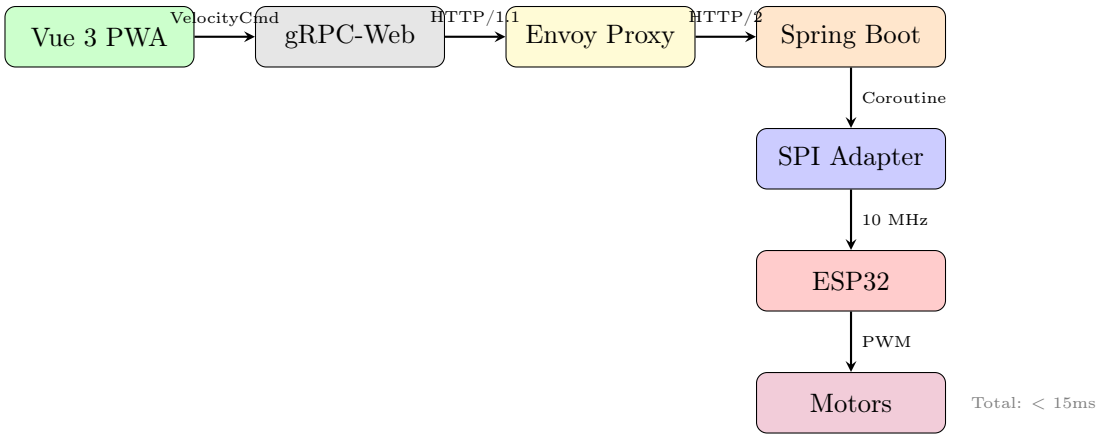


Figure 9: Motor Command Data Flow

13.2 SLAM Data Flow

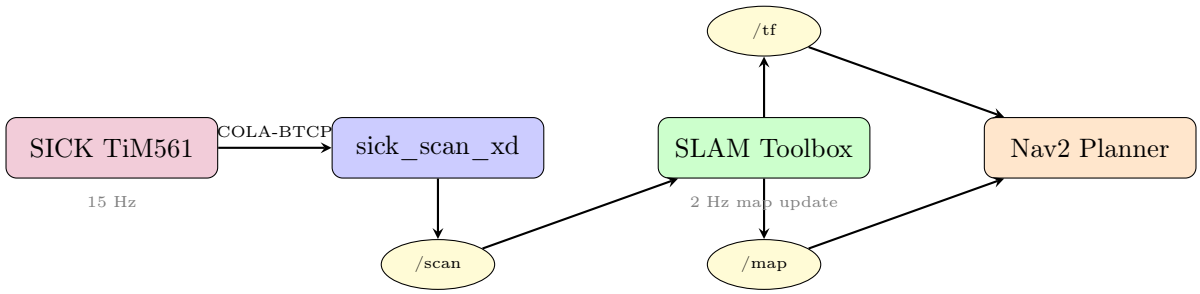


Figure 10: SLAM Data Flow (ROS 2 Topics)

13.3 Navigation Pipeline

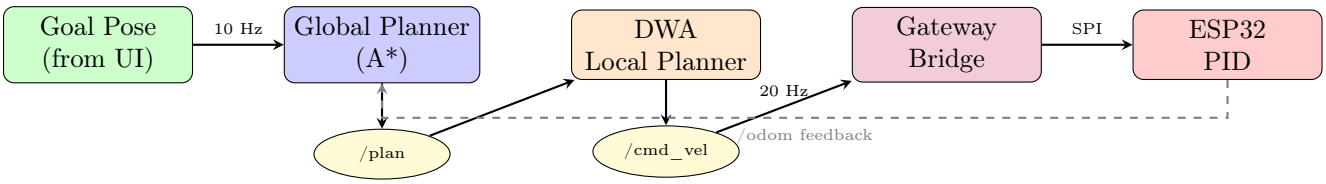


Figure 11: Navigation Pipeline Data Flow

13.4 Telemetry Aggregation Flow

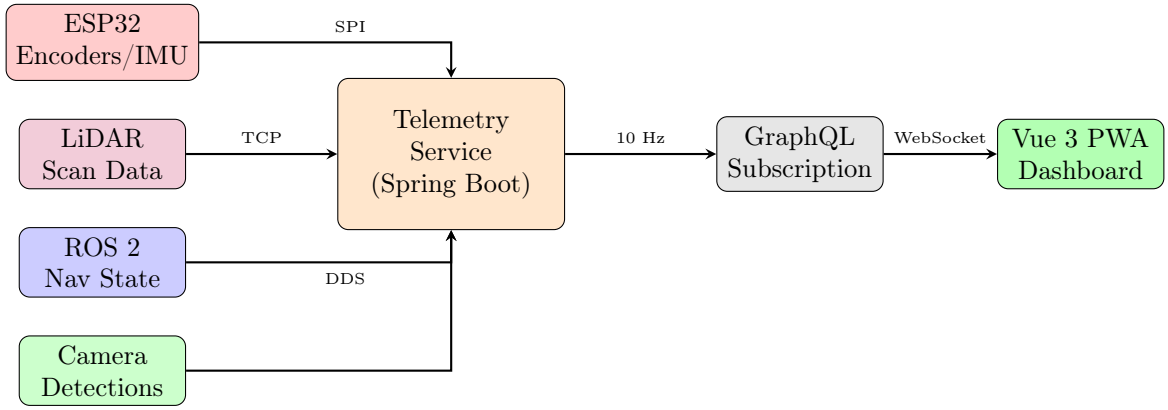


Figure 12: Telemetry Aggregation and Publishing

14 Deployment Configuration

14.1 Systemd Service

```
[Unit]
Description=STAR Robot Gateway
Documentation=https://github.com/locked-inc/star
After=network-online.target
```

```
Wants=network-online.target avahi-daemon.service

[Service]
Type=simple
User=star
Group=star
WorkingDirectory=/opt/star-gateway
ExecStart=/usr/bin/java \
    -Xmx256m \
    -Xms128m \
    -XX:+UseG1GC \
    -XX:MaxGCPauseMillis=50 \
    -Djava.security.egd=file:/dev/./urandom \
    -jar star-gateway.jar

# Restart policy
Restart=always
RestartSec=10
StartLimitInterval=300
StartLimitBurst=5

# Logging
StandardOutput=journal
StandardError=journal
SyslogIdentifier=star-gateway

# Environment
Environment="SPRING_PROFILES_ACTIVE=production"
Environment="JAVA_HOME=/usr/lib/jvm/java-17-openjdk"

# Security hardening
NoNewPrivileges=true
ProtectSystem=strict
ProtectHome=true
PrivateTmp=true
ReadWritePaths=/var/log/star-gateway /var/lib/star-gateway

# Resource limits
MemoryMax=512M
CPUQuota=80%

[Install]
WantedBy=multi-user.target
```

Listing 30: star-gateway/deployment/star-gateway.service

14.2 Avahi mDNS Service

```
<?xml version="1.0" standalone='no'?>
<!DOCTYPE service-group SYSTEM "avahi-service.dtd">
<service-group>
    <name replace-wildcards="yes">STAR Robot Gateway on %h</name>

```

```
    <!-- GraphQL HTTP endpoint -->
    <service>
        <type>_http._tcp</type>
        <port>8080</port>
        <txt-record>path=/graphql</txt-record>
        <txt-record>api=graphql</txt-record>
    </service>

    <!-- GraphQL WebSocket endpoint -->
    <service>
        <type>_ws._tcp</type>
        <port>8080</port>
        <txt-record>path=/graphql-ws</txt-record>
    </service>

    <!-- gRPC endpoint (via Envoy) -->
    <service>
        <type>_grpc._tcp</type>
        <port>8090</port>
        <txt-record>api=grpc-web</txt-record>
    </service>

    <!-- Health check endpoint -->
    <service>
        <type>_http._tcp</type>
        <port>8080</port>
        <txt-record>path=/actuator/health</txt-record>
        <txt-record>api=health</txt-record>
    </service>
</service-group>
```

Listing 31: star-gateway/deployment/avahi/star-robot.service

14.3 Envoy Proxy Configuration

```
admin:
  address:
    socket_address:
      address: 127.0.0.1
      port_value: 9901

static_resources:
  listeners:
    - name: grpc_web_listener
      address:
        socket_address:
          address: 0.0.0.0
          port_value: 8090
      filter_chains:
        - filters:
            - name: envoy.filters.network.http_connection_manager
              typed_config:
```

```
                "@type":
                   type.googleapis.com/envoy.extensions.filters.network.http_connectio
                codec_type: AUTO
                stat_prefix: grpc_web

                route_config:
                  name: local_route
                  virtual_hosts:
                    - name: grpc_services
                      domains: ["*"]
                      routes:
                        - match:
                            prefix: "/"
                          route:
                            cluster: grpc_backend
                            timeout: 60s

                      # CORS configuration
                      cors:
                        allow_origin_string_match:
                          - prefix: "*"
                        allow_methods: "GET, POST, OPTIONS"
                        allow_headers: >
                          content-type,
                          x-grpc-web,
                          x-user-agent,
                          grpc-timeout
                        max_age: "1728000"
                        expose_headers: >
                          grpc-status,
                          grpc-message

                http_filters:
                  # gRPC-Web to gRPC translation
                  - name: envoy.filters.http.grpc_web
                    typed_config:
                      "@type":
                         type.googleapis.com/envoy.extensions.filters.http.grpc_web.v3

                  # CORS handling
                  - name: envoy.filters.http.cors
                    typed_config:
                      "@type":
                         type.googleapis.com/envoy.extensions.filters.http.cors.v3.Cor

                  # Router
                  - name: envoy.filters.http.router
                    typed_config:
                      "@type":
                         type.googleapis.com/envoy.extensions.filters.http.router.v3.R

  clusters:
    - name: grpc_backend
      connect_timeout: 0.5s
```

```
      type: LOGICAL_DNS
      dns_lookup_family: V4_ONLY
      lb_policy: ROUND_ROBIN
      typed_extension_protocol_options:
        envoy.extensions.upstreams.http.v3.HttpProtocolOptions:
          "@type":
             type.googleapis.com/envoy.extensions.upstreams.http.v3.HttpProtocolOption
          explicit_http_config:
            http2_protocol_options: {}
      load_assignment:
        cluster_name: grpc_backend
        endpoints:
          - lb_endpoints:
              - endpoint:
                  address:
                    socket_address:
                      address: 127.0.0.1
                      port_value: 9090
```

Listing 32: /opt/envoy/envoy.yaml

15 Implementation Checklist

15.1 Phase 1: Infrastructure Setup

- ☐ Create `star-gateway/` Spring Boot project structure
- ☐ Create `proto/` shared protobuf definitions
- ☐ Set up Gradle build with gRPC + GraphQL plugins
- ☐ Add Envoy to Buildroot configuration
- ☐ Configure development environment (IDE, linting, testing)

15.2 Phase 2: Backend Core

- ☐ Implement `Esp32SpiAdapter` (Pi4J SPI communication)
- ☐ Implement `Esp32TcpAdapter` (TCP fallback)
- ☐ Implement `SickLidarAdapter` (COLA-B protocol)
- ☐ Implement domain services:
  - ☐ `MotorControlService`
  - ☐ `TelemetryService`
  - ☐ `NavigationService`
  - ☐ `SensorFusionService`
- ☐ Implement GraphQL controllers and schema
- ☐ Implement gRPC service for motor control

- ☐ Add Prometheus metrics and health endpoints
- ☐ Write unit tests for all services

15.3 Phase 3: Frontend PWA

- ☐ Copy UI-Demo to `star-ui/`
- ☐ Add PWA dependencies and Vite PWA plugin
- ☐ Create Apollo Client with offline persistence
- ☐ Create gRPC-Web client with proto compilation
- ☐ Replace `useMockData` hook with `useRobotConnection`
- ☐ Implement IndexedDB storage layer
- ☐ Create service worker with Workbox
- ☐ Add offline UI indicators
- ☐ Test PWA installation and offline mode

15.4 Phase 4: ESP32 Updates

- ☐ Create `star_module_tcp` library
- ☐ Implement TCP server with JSON parsing
- ☐ Add SPI slave command handler (if not present)
- ☐ Define binary command protocol
- ☐ Integrate TCP server into `main.c`
- ☐ Test SPI + TCP redundancy

15.5 Phase 5: Deployment

- ☐ Create systemd service file
- ☐ Create Avahi mDNS service file
- ☐ Configure Envoy proxy
- ☐ Update Buildroot to include:
  - ☐ OpenJDK 17
  - ☐ Envoy proxy
  - ☐ Gateway JAR
- ☐ Create deployment scripts
- ☐ Integration testing on hardware

15.6 Phase 6: Documentation

- ☐ Update `GEMINI.md` with new architecture
- ☐ Create API documentation (GraphQL schema + gRPC)
- ☐ Update hardware testing guide
- ☐ Create troubleshooting guide
- ☐ Document deployment procedures

16 Technology Summary

Layer	Technology	Purpose
Frontend (PWA)		
UI Framework	Vue 3 + TypeScript + Vite 6	Reactive UI with Composition API
PWA	Workbox 7 + Vite PWA	Offline capability, installability
GraphQL Client	Vue Apollo 4 + Apollo Client	Data fetching, caching, subscriptions
gRPC Client	grpc-web 0.15	Low-latency motor commands
Offline Storage	IndexedDB (idb 8)	Full telemetry history offline
Styling	Tailwind CSS 4	Utility-first responsive design
Backend (Gateway)		
Framework	Spring Boot 3.2	Production-ready JVM framework
Language	Kotlin 1.9	Modern JVM language with coroutines
GraphQL Server	Spring GraphQL	Queries, mutations, subscriptions
gRPC Server	grpc-spring-boot-starter 3.1	Binary RPC for motor control
Hardware Access	Pi4J 2.6+ (GpioD)	SPI/GPIO access on RPi5 RP1 chip
Metrics	Micrometer + Prometheus	Health monitoring, observability
Infrastructure		
Proxy	Envoy 1.29	gRPC-Web to gRPC translation
Service Discovery	Avahi (mDNS)	Zero-config `star-robot.local`
Process Manager	systemd	Service lifecycle, auto-restart
OS	Buildroot 2025.02 LTS	Minimal Linux for RPi5
Firmware (ESP32)		
Framework	ESP-IDF 5.x	Official Espressif SDK
RTOS	FreeRTOS	Real-time task scheduling
Communication	SPI + TCP/WiFi	Dual-channel redundancy
Architecture	STAR Firmware (DIP)	Modular, testable embedded code

Table 10: Complete Technology Stack

17 References and Resources

17.1 Protocol Documentation

- gRPC Official Documentation: https://grpc.io/docs/
- GraphQL Specification: https://spec.graphql.org/
- gRPC-Web GitHub: https://github.com/grpc/grpc-web
- Spring GraphQL: https://spring.io/projects/spring-graphql

17.2 Framework Documentation

- Spring Boot 3: https://docs.spring.io/spring-boot/docs/3.2.x/reference/
- Vue Apollo: https://v4.apollo.vuejs.org/
- Workbox: https://developer.chrome.com/docs/workbox/
- Pi4J: https://pi4j.com/documentation/

17.3 Research References

- Hasura: "GraphQL vs gRPC" - https://hasura.io/learn/graphql/intro-graphql/graphql-vs-grpc/
- Stack Overflow Blog: "When to use gRPC vs GraphQL" - https://stackoverflow.blog/2022/11/28/when-to-use-grpc-vs-graphql/
- Ambassador Labs: "Protobuf vs JSON Performance" - https://www.getambassador.io/blog/protobuf-vs-json
- Postman: "gRPC vs GraphQL" - https://blog.postman.com/grpc-vs-graphql/

17.4 Build Tools

- Vite: https://vitejs.dev/
- Gradle: https://gradle.org/
- PlatformIO: https://platformio.org/